Comestra

Cozinha, balcão e entrega no mesmo compasso.

Guia completo para instalar, migrar, configurar e operar a plataforma SaaS multi-loja de delivery e gestão para restaurantes: superpainel, lojas, planos e assinaturas, painel da loja, vitrine, segurança e operação.

SaaS multi-tenant **API NestJS 11** **PostgreSQL + TypeORM (migrations)** **Web React 19 + Vite 7**
Mercado Pago por loja **Testes Jest + Playwright** **Docker**

Atualizado em 01/11/2026 · versão da API 0.0.1 · escopo desta fase: backend e frontend web (o app mobile não foi alterado)
Caminhos relativos à raiz do repositório (`backend/` , `frontend-web/` , `frontend-mobile/` , `e2e/` , `docs/`).
Público: desenvolvedores, operação da plataforma e equipe de implantação. Referência técnica detalhada em `docs/saas/arquitetura.md` e `docs/saas/operacao.md` .

Sumário

1. Visão geral
2. Checklist de execução rápida
3. Estrutura do repositório
4. Pré-requisitos
5. Variáveis de ambiente
6. Banco de dados e migrations
7. Superadministrador e contas de demonstração
8. Isolamento entre lojas
9. Superpainel
10. Planos e assinaturas
11. Painel da loja
12. Vitrine e conta do cliente
13. Pedidos: regras, visão por perfil e transições
14. Pagamentos do delivery
15. Segurança
16. Mapa de endpoints da API
17. API: comandos e comportamento global
18. Frontend web: comandos, rotas e sessão
19. App mobile (fora do escopo)
20. Testes
21. Produção
22. Integrações externas
23. Migração a partir do TorresBurgers
24. Limitações conhecidas e pendências
25. Troubleshooting

1. Visão geral

A **Comestra** é uma plataforma SaaS **multi-tenant** de delivery e gestão para restaurantes. Uma única instalação atende muitos estabelecimentos: cada loja tem equipe, cardápio, pedidos, pagamentos, configurações de entrega, identidade visual e assinatura próprios, isolados no servidor. A Comestra (dona da plataforma) administra tudo pelo **superpainel**. Os **clientes** têm conta global e compram em qualquer loja.

Frente	Para quem	O que oferece
Diretório e vitrines (web)	Clientes	Diretório em `/` ; vitrine de cada loja em `/loja/<slug>` com cardápio, carrinho por loja, checkout (PIX, cartão ou na entrega) e Meus Pedidos de todas as lojas.
Painel da loja (web)	Dono, atendentes, cozinha e entregadores	Dashboard, quadro de pedidos por etapa (com o número do pedido na loja e o alerta de pagamento estornado), despacho, cardápio, equipe e convites, clientes (com bloqueio por loja), configurações (entrega por bairro e/ou faixa de CEP, com recusa fora da área), credenciais Mercado Pago, assinatura (com o medidor de armazenamento) e Minha conta.
Superpainel (web, `/platform`)	Superadministrador da Comestra	Lojas, usuários, planos (com limite de armazenamento), assinaturas e faturas, financeiro, auditoria, configurações (inclusive e-mail transacional e manutenção, com limpeza automática) e sessão de suporte somente leitura.
App mobile (Expo)	Clientes e equipe	Fora do escopo desta fase: continua funcionando pelas rotas antigas, na **loja padrão**.

Camada	Tecnologias	Responsabilidade
API (`backend/`)	NestJS 11, TypeScript, TypeORM 0.3 (migrations), PostgreSQL, Passport JWT, bcrypt, class-validator, Swagger, Sharp, SDK do Mercado Pago, nodemailer (SMTP)	Isolamento entre lojas, superpainel, planos e assinaturas, regras de negócio, preço no servidor, pagamentos por loja, auditoria e segurança.
Web (`frontend-web/`)	React 19, Vite 7, TailwindCSS 4, React Router 7, Axios, Recharts, Lucide/Iconify, SDK JS v2 do Mercado Pago	Diretório, vitrines, conta do cliente, painel da loja e superpainel.
Mobile (`frontend-mobile/`)	React Native 0.81, Expo SDK 54	Loja do cliente e painel da equipe na loja padrão (sem alterações nesta fase).
Docker (opcional)	`docker-compose.yml` : PostgreSQL 18, API (Node 22) e Nginx	Sobe tudo com um comando, site e API na mesma origem (`http://localhost:8080` , API em `/api`).
Testes (`e2e/`)	Playwright (API + Google Chrome)	Ponta a ponta em ambiente isolado, inclusive isolamento entre lojas, superpainel (também no navegador), cobrança, sessão em cookie e conta.

Perfis de acesso

Perfil	Resumo
`SUPER_ADMIN` (global)	Superpainel inteiro. Não opera lojas nem faz pedidos; pode abrir uma sessão de suporte somente leitura numa loja. Nunca concedido por HTTP (só pelo comando `npm run platform:superadmin`).
`ADMIN` (Administrador)	Tudo da própria loja: Dashboard, pedidos, cardápio, equipe e convites, clientes, lixeira, configurações e assinatura.
`EMPLOYEE` (Atendente)	Todos os pedidos da loja, despacho, cancelamento, abrir/fechar a loja e clientes (leitura).
`KITCHEN` (Cozinha)	Só os pedidos novos e em preparo, sem dados de contato do cliente.
`COURIER` (Entregador)	Entregas prontas sem responsável e as próprias entregas.
`CLIENT` (Cliente)	Conta global: compra em qualquer loja, endereços e dados da conta.

As permissões são aplicadas pela API (guards de loja, de perfil e da plataforma), e não apenas escondidas nas telas. A loja da equipe vem do usuário autenticado e a da vitrine vem do slug da rota: nenhum identificador de loja enviado pelo navegador concede acesso. Pagamentos do delivery (cliente → loja) e cobrança da assinatura SaaS (loja → Comestra) usam fluxos, tabelas, credenciais e webhooks separados.

---

2. Checklist de execução rápida

Caminho curto para subir tudo localmente, do zero. Os comandos valem para Linux e para o PowerShell do Windows, exceto onde indicado.

Atalho com Docker. Com Docker Desktop (ou Docker Engine com Compose v2): `cp .env.docker.example .env.docker`, preencha `POSTGRES_PASSWORD`, `JWT_SECRET` e `CREDENTIALS_ENCRYPTION_KEY` (os comandos de geração estão no arquivo) e rode `docker compose up -d --build`. O site abre em `http://localhost:8080` (API em `/api`), no modo demonstração. Guia completo: `docs/saas/docker.md`. Os passos abaixo são a instalação sem Docker.

1. Clonar o projeto e criar o banco

```
git clone <URL_DO_REPOSITORIO> comestra
cd comestra

# Windows (PowerShell) — troque 18 pela versão instalada do PostgreSQL
& "C:\Program Files\PostgreSQL\18\bin\createdb.exe" -U postgres comestra
# Linux
sudo -u postgres createdb comestra
```

2. Configurar e subir a API

```
cd backend
cp .env.example .env        # Windows: Copy-Item .env.example .env
# edite o .env: NODE_ENV=development e DB_PASSWORD (deixe DB_SYNCHRONIZE vazio = false)
npm install
npm run start:dev
```

A API aplica as 13 migrations e cria a loja **Lanchonete Demo** (`/loja/demo` , loja padrão, aberta) com a equipe demo e um cliente demo. Confira: `http://localhost:3000/health` responde `{"status":"ok"}` , o log mostra `API Comestra escutando em 0.0.0.0:3000 (todas as interfaces: acessível pela rede local)` e o Swagger fica em `http://localhost:3000/api` . Sem `JWT_SECRET` , a primeira subida em development cria um segredo aleatório desta máquina em `backend/.temp/dev-jwt-secret` (fora do Git).

3. Criar o seu superadmin

```
npm run platform:superadmin -- --email=voce@exemplo.com --name="Seu Nome"   # a senha aparece uma única vez
```

4. Subir o site, os painéis e o superpainel

```
cd ../frontend-web
npm install
npm run dev
```

Diretório em `http://localhost:5173` , vitrine demo em `/loja/demo` , login do cliente em `/signin` , equipe e superadmin em `/login` (o superadmin vai para `/platform`).

5. Entrar com as contas de demonstração

Perfil	E-mail	Senha	Onde entrar
Administrador	`admin@comestra.com.br`	`Admin@123`	`/login` → Dashboard
Atendente	`atendente@comestra.com.br`	`Atendente@123`	`/login` → Pedidos
Cozinha	`cozinha@comestra.com.br`	`Cozinha@123`	`/login` → Monitor da Cozinha
Entregador	`entregador@comestra.com.br`	`Entregador@123`	`/login` → Painel de Entregas
Cliente	`cliente@comestra.com.br`	`Cliente@123`	`/signin` → diretório
Superadmin demo	`superadmin@comestra.com.br`	`Superadmin@123`	Só existe em `test` ou com `SEED_DEMO_SUPERADMIN=true` / `npm run seed -- --with-superadmin`

Senhas públicas. Antes de expor uma API de desenvolvimento pela rede ou por túnel, desligue `SEED_DEMO_USERS`, troque ou exclua as contas demo e confira o superadmin demo (seção 7).

6. Criar uma segunda loja

No superpainel: **Lojas** → **Nova loja**. A loja nasce fechada, com a entrega desligada, a assinatura do plano escolhido e um convite do responsável (o link aparece uma vez; com SMTP, também vai por e-mail). Abra o link em `/convite/:token`, defina a senha, configure a loja, cadastre o cardápio e abra a loja.

3. Estrutura do repositório

```
comestra/
├── backend/                      API REST multi-tenant (NestJS)
│   ├── src/tenancy/              resolução da loja, guards, situação da loja × capacidades (store-
access.ts)
│   ├── src/platform/             superpainel (lojas, usuários, suporte, configurações, métricas) e
cli/ do superadmin
│   ├── src/billing/              planos, assinaturas, faturas, limites, rotina e webhook da cobrança
SaaS
│   ├── src/audit/                auditoria (metadados sanitizados)
│   ├── src/auth/                 login, JWT, sessões registradas (jti), sessão do site em cookie,
logout, recuperação de senha, suporte somente leitura
│   ├── src/account/              minha conta: senha, confirmação e troca de e-mail ("Não fui eu")
│   ├── src/invitations/          convites da equipe
│   ├── src/users/                clientes e equipe, responsável da loja, avatar, regras de acesso
│   ├── src/products/             cardápio e adicionais por loja
│   ├── src/orders/               pedidos, preço no servidor, visão por perfil, transições, dashboard,
relatórios
│   ├── src/payment/              Mercado Pago por loja, sandbox, finalização, estornos, webhook e
conciliação
│   ├── src/store/                status, configurações, credenciais, clientes, diretório e perfil
público
│   ├── src/addresses/            endereços, busca de CEP e conferência do bairro pelo CEP
│   ├── src/promotions/           banner do dia
│   ├── src/notifications/        e-mails em texto e HTML com a marca da loja (driver de log ou SMTP)
e links
│   ├── src/common/               uploads e cota de armazenamento, limites (RateLimitService),
criptografia, manutenção, validações
│   ├── src/config/               brand.ts, env.ts, jwt.ts, app-setup.ts (CORS, validação e
cabeçalhos)
│   ├── src/database/             data source, migrations, seeds (loja e contas de demonstração)
│   ├── scripts/                  scripts opcionais de dados (atuam só na loja padrão)
│   ├── Dockerfile
│   └── .env.example
├── frontend-web/                 diretório, vitrines, painel da loja e superpainel (React + Vite)
│   ├── src/                      pages/ (Directory, Client, Auth, painel, Platform), components/ui
(kit de UI),
│   │                             contexts/, layouts/, routes/, services/ (platform/*), hooks/,
types/, utils/
│   ├── vite.config.ts            CSP do build, cabeçalhos do vite preview e pedaços react/charts
│   └── Dockerfile, nginx.conf    imagem do site (Nginx: mesma origem, /api → API, logs sem tokens)
├── frontend-mobile/              app do cliente e da equipe (Expo) — fora do escopo desta fase
├── e2e/                          testes ponta a ponta (Playwright):
api/{identity,commerce,tenancy,platform,billing,security}, web/{client,admin,platform,multistore}
├── docker-compose.yml, .env.docker.example   Docker: banco, API e site
└── docs/                         saas/ (arquitetura.md, operacao.md, docker.md, diagnostico.md) e
este guia (HTML e PDF)
```

Cada app tem um `.env.example` comentado; o `.env` real é ignorado pelo Git. O setup deste guia usa PostgreSQL instalado localmente e `npm`; a alternativa com Docker (`docker-compose.yml`, `.env.docker` da raiz, também ignorado pelo Git) está em `docs/saas/docker.md`.

4. Pré-requisitos

Ferramenta	Versão	Observações
Node.js e npm	22+ (LTS)	Necessário para a API, a web e a suíte E2E. Cada app usa o próprio `package-lock.json`.
PostgreSQL	14+ (testado no 18)	Banco padrão: `comestra`. O schema vem das migrations. Use `pg_dump` / `pg_restore` para os backups.
Google Chrome	Atual	Só para a suíte E2E (o Playwright usa o Chrome instalado).
Docker (opcional)	Compose v2	Só para subir tudo em containers (`docs/saas/docker.md`); cerca de 3 GB livres e a porta 8080.
Expo Go / OpenSSH	SDK 54 / recente	Só para o app mobile (fora do escopo) e o túnel `npm run tunnel`.

5. Variáveis de ambiente

API: `backend/.env`

Variáveis definidas no próprio ambiente (servidor, Docker, CI) têm prioridade sobre o `.env`. Valores vazios usam o padrão. A lista completa e comentada está em `backend/.env.example`.

Variável	Padrão	Descrição
`NODE_ENV`	vazio = produção	`development` / `test` liberam as facilidades de desenvolvimento. Qualquer outro valor, ou nenhum, segue as regras de produção (falha fechada).
`PORT` / `HOST`	`3000` / `0.0.0.0`	Porta e interface de escuta. `0.0.0.0` = todas as interfaces (rede local, app mobile no celular); `127.0.0.1` = só este computador (túnel ou proxy na mesma máquina). O log mostra `escutando em <host>:<porta> (<alcance>)`.
`DB_HOST` / `DB_PORT` / `DB_USERNAME` / `DB_PASSWORD` / `DB_DATABASE`	`localhost` / `5432` / `postgres` / vazio / `comestra`	Conexão com o PostgreSQL.
`DB_MIGRATIONS_RUN`	`true`	Aplica as migrations pendentes na subida. Várias instâncias: `false` nelas e `migration:run:prod` no deploy.
`DB_SYNCHRONIZE`	`false`	Synchronize do TypeORM. Mantenha `false` em todos os ambientes (seção 6).
`DB_ALLOW_DESTRUCTIVE_REVERT`	vazio	Libera o `down()` destrutivo das migrations `MultiTenantFoundation` e `OrderNumbersAndCustomerBlocks` (só com backup).
`DB_POOL_SIZE` / `DB_STATEMENT_TIMEOUT_MS` / `DB_POOL_ACQUIRE_TIMEOUT_MS`	`10` / `60000` / `30000`	Conexões do pool da API; tempo máximo de cada consulta (`0` desliga; não vale para as migrations nem para o CLI); espera máxima por uma conexão livre (`0` = sem limite).
`INITIAL_STORE_NAME` / `INITIAL_STORE_SLUG`	"Loja principal" / `loja-principal`	Loja que recebe os dados na migração de uma instalação de loja única.
`DEFAULT_STORE_SLUG`	vazio	Loja das rotas antigas sem slug (app mobile). Vazio = a escolhida no superpainel.
`JWT_SECRET` / `JWT_EXPIRES_IN`	development: aleatório por máquina (`.temp/dev-jwt-secret`, caminho em `JWT_DEV_SECRET_FILE`); test: fixo / `7d`	Assina os tokens. Obrigatório em produção (32+ caracteres; a API não sobe sem ele). `JWT_EXPIRES_IN` é a validade do token bearer (app mobile, sessões antigas, suporte).

Variável	Padrão	Descrição
`JWT_LEGACY_ISSUED_BEFORE`	vazio	Instante ISO 8601 do deploy que aplicou a `AuthSessions` (ex.: `2026-11-01T12:00:00Z`): tokens sem `jti` emitidos antes valem até ele + `JWT_EXPIRES_IN`. Vazio, em produção: nenhum é aceito (todos entram de novo); em dev/test, todos até expirar. `none` recusa todos.
`AUTH_WEB_COOKIE` / `JWT_ACCESS_EXPIRES_IN` / `AUTH_SESSION_MAX_AGE`	`true` / `15m` / = `JWT_EXPIRES_IN`	Sessão do site em cookie (seção 15): liga/desliga (imediato), validade do token de acesso (1 a 60 min) e validade total da sessão (5 min a 90 dias; a renovação não estende).
`AUTH_REFRESH_GRACE_SECONDS` / `AUTH_COOKIE_ORIGINS` / `AUTH_COOKIE_SAMESITE`	`30` / vazio / `strict`	Tolerância do refresh anterior (0 a 120 s); origens que recebem credenciais (vazio = `CORS_ORIGINS` explícito + origem de `APP_WEB_URL`; em produção sem nenhuma, o modo cookie fica desligado); `strict` ou `lax`.
`ACCOUNT_SECURITY_MODE`	`compat`	Verificação e troca de e-mail. `strict` ainda não existe nesta versão (só gera aviso na subida).
`CREDENTIALS_ENCRYPTION_KEY`	fixa só em dev/test	Chave AES-256 (32 bytes em base64 ou hex) das credenciais Mercado Pago das lojas. Obrigatória em produção; trocá-la torna as credenciais gravadas ilegíveis.
`APP_NAME` / `APP_WEB_URL`	`Comestra` / `http://localhost:5173`	Nome da plataforma; endereço do site nos links de convite, de redefinição e dos e-mails de conta.
`PUBLIC_API_URL`	vazio	Endereço público da API (ex.: `https://api.suaempresa.com.br`; no Docker, `http://localhost:8080/api`): base do logo das lojas nos e-mails. Vazio = e-mails sem logo (monograma).
`MAIL_DRIVER`	`log`	E-mail transacional: `log` (nada é enviado; em produção só destinatário e assunto vão para o log, com aviso na subida) ou `smtp` (seção 17).
`SMTP_HOST` / `SMTP_PORT` / `SMTP_SECURE`	vazio / `587` (`465` com `SMTP_SECURE=true`) / só na porta 465	Servidor SMTP, obrigatório com `smtp`. Desligado, `SMTP_SECURE` usa STARTTLS, obrigatório quando há usuário e senha.
`SMTP_USER` / `SMTP_PASSWORD`	vazio	Opcionais, sempre juntos. Nunca aparecem em respostas, logs, auditoria nem no superpainel.
`MAIL_FROM`	vazio	Remetente, obrigatório com `smtp`: `"Nome <endereco@dominio>"` ou só o endereço (entre aspas no `.env`).
`UPLOADS_DIR`	`./uploads`	Pasta das imagens (servidas em `/uploads`, com cache de 30 dias). Produção: volume persistente.
`UPLOADS_CLEANUP_INTERVAL_HOURS`	vazio (desligada)	Limpeza automática dos arquivos sem uso, em horas (ex.: `24`; decimais aceitos; `0` / `off` desliga). Apaga só imagens sem uso há mais de 24 h.

Variável	Padrão	Descrição
`PLATFORM_INSIGHTS_CACHE_MS`	`30000`	Cache do dashboard e do financeiro do superpainel (`0` desliga).
`SWAGGER_ENABLED`	dev/test: ligado	Documentação em `/api` ; desligada em produção.
`CORS_ORIGINS`	vazio = qualquer origem	Domínios do site e do painel, separados por vírgula. O app mobile não envia `Origin` .
`TRUST_PROXY`	vazio	IP real atrás de proxy/túnel: número de proxies (ex.: `1` ; `2` com um proxy HTTPS na frente do Nginx do Docker) ou lista de IPs/sub-redes deles. `true` vale como `1` , com aviso (com `true` o cliente escolheria o próprio IP). Use `10.0.0.0/8` , nunca `::ffff:10.0.0.0/8` . Sem ele, os limites por IP viram um balde único; a API avisa no log.
`RATE_LIMIT_ENABLED` / `RATE_LIMIT_<NOME>`	ligado / ver tabela da seção 15	Limites de tentativas e de abuso, formato `limite/segundos` , em memória e por instância.
`SEED_DEMO_USERS`	dev/test: ligado	Loja demo (banco sem lojas), equipe demo e cliente demo. Nunca em produção.
`SEED_DEMO_SUPERADMIN`	test: ligado; development: desligado	Superadmin de demonstração (senha pública). Nunca em produção.
`SUPERADMIN_PASSWORD`	vazio	Senha do comando `platform:superadmin` no lugar da aleatória (12+ caracteres).
`MP_ACCESS_TOKEN` / `MP_PUBLIC_KEY`	vazio	Conta Mercado Pago usada só pela loja padrão sem credencial própria.
`PAYMENT_SANDBOX` / `PIX_SANDBOX_APPROVAL_SECONDS`	automático / `5`	Sandbox só em dev/test. Em produção nunca liga.
`PIX_EXPIRATION_MINUTES`	`30`	Validade do QR Code PIX (de 30 min a 30 dias).
`MP_NOTIFICATION_URL`	vazio	URL pública de `POST /payment/webhook` . Sem ela, a conciliação (60 s) confirma os pagamentos.
`SAAS_BILLING_PROVIDER`	`MANUAL`	`MANUAL` (faturas registradas pelo superadmin) ou `MERCADO_PAGO` (assinaturas recorrentes).
`MP_SAAS_ACCESS_TOKEN` / `MP_SAAS_WEBHOOK_SECRET` / `SAAS_BILLING_BACK_URL`	vazio	Conta Mercado Pago da Comestra (nunca a de uma loja), segredo do webhook `POST /billing/webhook/mercadopago` e página de retorno.
`SAAS_BILLING_INTERVAL_MS`	`300000`	Intervalo da rotina das assinaturas (mínimo 10000).
`STORE_TIME_ZONE`	`America/Sao_Paulo`	Fuso das lojas que não têm o próprio.
`DELIVERY_ZONES_JSON` / `DELIVERY_ZONES_FILE`	vazio	Faixas de entrega da loja padrão (as demais lojas configuram no painel).

Variável	Padrão	Descrição
`FOOTBALL_DATA_TOKEN`	vazio	Token de football-data.org para o banner de jogos (opcional).
`CEP_LOOKUP_FAKE`	vazio	Só com `NODE_ENV=test` (ignorada com aviso nos outros): mapa JSON de CEPs no lugar de ViaCEP/BrasilAPI. Nunca em produção.

Booleanos aceitam `true/false`, `1/0`, `yes/no`, `sim/não` e `on/off`. Um valor inválido usa o padrão e gera aviso no log.

Web: `frontend-web/.env`

Variável	Padrão	Descrição
`VITE_API_URL`	`http://localhost:3000`	Endereço da API, sem barra no final (ou relativo, como `/api`, com site e API na mesma origem, como no Docker). Também entra na Content-Security-Policy do build e nos cabeçalhos do `vite preview`: o build de produção precisa da URL real.
`VITE_APP_NAME`	`Comestra`	Nome da plataforma exibido.
`VITE_MP_PUBLIC_KEY`	chave de teste	Chave pública de compatibilidade; cada loja usa a chave pública do próprio perfil.
`VITE_AUTH_TRANSPORT`	`auto`	`auto`: sessão em cookie quando a API permite; `bearer`: desliga o modo cookie no site (kill switch, exige build novo).

Variáveis `VITE_*` entram no JavaScript público do site. Nunca coloque segredos nelas.

Testes E2E (`e2e/`)

Variável	Padrão	Descrição
`E2E_API_PORT` / `E2E_WEB_PORT`	`3001` / `5174`	Portas do ambiente de teste.
`E2E_DB_DATABASE`	`comestra_e2e`	Banco de teste (precisa terminar em `_e2e`).
`E2E_UPLOADS_DIR`, `E2E_OUTPUT_DIR`, `E2E_REPORT_DIR`, `E2E_SKIP_BUILD`	pastas padrão	Execuções paralelas isoladas (seção 20).
`DB_HOST`, `DB_PORT`, `DB_USERNAME`, `DB_PASSWORD`	`localhost`, `5432`, `postgres`, vazio	Conexão usada para recriar o banco de teste.
`E2E_CREDENTIALS_ENCRYPTION_KEY`	chave do ambiente ou a de desenvolvimento	Chave com que o spec de manutenção monta credenciais no formato antigo; precisa ser a da API de teste (senão o teste é pulado).
`E2E_ACCOUNT_SECURITY_MODE`	`compat`	`ACCOUNT_SECURITY_MODE` da API de teste (reservado para a futura execução em `strict`).

6. Banco de dados e migrations

O schema vem das migrations TypeORM em `backend/src/database/migrations` (registradas em `index.ts`), aplicadas na subida da API (`DB_MIGRATIONS_RUN`) ou pelos comandos abaixo. Cada migration roda na própria transação, exceto a `PerformanceIndexes` (índices com `CONCURRENTLY`).

Migration	O que faz
`1791764966404-BaselineSchema`	Schema exato da versão de loja única. Num banco existente não faz nada. Irreversível.
`1791765597480-MultiTenantFoundation`	Tabelas da plataforma; `store_id` nas tabelas operacionais; migração dos dados para a loja inicial; FKs, índices, CHECKs e triggers.
`1791773437187-TenancyIndexes`	Índices por loja/cliente e únicos parciais (convite pendente por loja + e-mail; fatura por período). Idempotente (`IF NOT EXISTS`).
`1792299658658-MaintenanceJobs`	Tabela `maintenance_jobs` (manutenção em segundo plano, uma execução por vez).
`1792800000000-PlatformIntegrity`	`subscriptions.suspended_by` (preenchida pelo histórico), CHECK do papel global, triggers de mesma loja, `store_id` imutável nas tabelas da plataforma e equipe sem troca de loja.
`1792963851057-OrderNumbersAndCustomerBlocks`	`orders.number` com **renumeração dos pedidos existentes por loja**, na ordem de criação; contador `store_order_counters`; gatilhos que numeram todo pedido novo (sem buraco nem repetição) e impedem alterar o número; tabela `store_customer_blocks`. Tolera coluna e índice criados pelo synchronize num banco de desenvolvimento. Com bloqueios gravados, o `down()` recusa reverter (salvo com `DB_ALLOW_DESTRUCTIVE_REVERT=true`).
`1792964108700-AuthSessions`	Tabela `auth_sessions` (uma linha por token de sessão, `jti` único). Tokens emitidos antes dela só valem com `JWT_LEGACY_ISSUED_BEFORE`. O `down()` encerra as sessões de todas as contas.
`1793476392120-AddressZoneLocation`	Bairro, cidade e UF conferidos pelo CEP em `addresses` (colunas opcionais e CHECKs, sem backfill: os endereços antigos são conferidos no primeiro orçamento de entrega).
`1793477263977-PlanStorageLimit`	`plans.max_storage_mb` (nulo = ilimitado; os planos existentes ficam ilimitados).
`1793485300000-StoreSlugHistoryAndCancelOrigin`	`subscriptions.cancel_scheduled_by` (preenchida pelo histórico) e `store_slug_history` (slugs antigos reservados, preenchida pela auditoria).
`1793490000000-PerformanceIndexes`	Índices de desempenho com `CREATE INDEX CONCURRENTLY`, fora de transação (se falhar no meio, rode de novo). Os índices de busca de contas só com a extensão `pg_trgm` disponível.
`1793491000000-AccountEmailVerification`	`users.email_verified_at` e a tabela `account_email_tokens` (links de confirmação, troca e "Não fui eu"; só o hash).

Migration	O que faz
`1793492000000-AuthSessionRefresh`	Colunas da sessão do site em cookie em `auth_sessions` (transporte, seletor e hashes do refresh, geração, sessão-mãe do suporte). As sessões existentes viram `BEARER`.

Comandos (em `backend/`)

Comando	Para quê
`npm run migration:show`	Estado: `[X]` aplicada, `[ ]` pendente.
`npm run migration:run`	Aplica as pendentes (código-fonte, ts-node).
`npm run migration:revert`	Desfaz a última aplicada (repita para voltar mais).
`npm run migration:check`	Falha se as entidades divergem do banco (CI ou cópia do banco migrado).
`npm run migration:generate -- src/database/migrations/Nome`	Gera uma migration nova a partir das entidades (revise e registre em `index.ts`).
`npm run migration:run:prod` / `migration:show:prod`	O mesmo a partir do build (`dist/`).
`npx typeorm -d dist/database/data-source.js migration:revert`	Reverter a partir do build.

> `DB_SYNCHRONIZE=false` **em todos os ambientes.** Com `true`, o TypeORM roda depois das migrations e cria por fora delas os índices, colunas e CHECKs que as entidades declaram: o banco fica diferente do de produção, uma migration esquecida passa despercebida e o synchronize pode até apagar colunas. Bancos de desenvolvimento antigos (subidos com `true`) podem já ter índices que as migrations criam: a `TenancyIndexes` tolera isso.

Backup e restauração

```
pg_dump -U postgres -Fc comestra > comestra-AAAAMMDD.dump          # backup
createdb -U postgres -T comestra comestra_backup_AAAAMMDD          # cópia no mesmo servidor (sem conexões abertas)
createdb -U postgres comestra_restaurado
pg_restore -U postgres -d comestra_restaurado comestra-AAAAMMDD.dump   # restauração
```

Inclua também a pasta de imagens (`UPLOADS_DIR`) no backup.

Atualizar uma instalação existente

1. Backup do banco (`pg_dump`, sempre antes de migrations novas) e das imagens.
2. `DB_SYNCHRONIZE=false` (ou vazio) no `backend/.env`.
3. Vindo de uma versão sem as sessões registradas (`AuthSessions`): defina `JWT_LEGACY_ISSUED_BEFORE` com o instante do deploy (ex.: `2026-11-01T12:00:00Z`) para os tokens antigos valerem até expirar; sem ela, em produção, **todos (site, app mobile e equipe) entram de novo**.
4. Atualize o código e rode `npm install` em `backend/` e `frontend-web/`.

5. Suba a API (as pendentes são aplicadas na subida) ou rode `npm run migration:run`; confira com `migration:show` (13 com `[X]`) e `migration:check`. Quem já tinha as 7 primeiras recebe 6 novas, todas aditivas.
6. Se havia credenciais Mercado Pago gravadas, rode **Regravar credenciais** em Superpainel → Configurações → Manutenção (formato antigo → atual).

> **O número do pedido muda nas telas.** A `OrderNumbersAndCustomerBlocks` renumera os pedidos existentes **por loja** (1, 2, 3... em cada loja, na ordem de criação). Antes, as telas mostravam `Pedido #<id>`, o id global: o pedido que era "Pedido #1532" pode virar "Pedido #87". Conversas, comprovantes e anotações antigas que citavam o id deixam de bater com o número exibido; avise as lojas antes de atualizar. O `id` não muda: continua nas rotas da API, nos pagamentos e no app mobile.

A `PlatformIntegrity` recusa subir se alguma conta com papel global também for da equipe de uma loja: a mensagem lista as contas; remova o papel (`npm run platform:superadmin -- --email=<e-mail> --revoke`) ou tire a conta da equipe e rode de novo. Uma migration que falha é desfeita e a API não sobe; as anteriores continuam aplicadas (a `PerformanceIndexes`, fora de transação, completa os índices que faltam na próxima execução).

Da loja única para o SaaS

1. Opcional: `INITIAL_STORE_NAME="Minha Lanchonete"` e `INITIAL_STORE_SLUG=minha-lanchonete`.
2. A `MultiTenantFoundation` cria a loja inicial com o status e as mensagens de `store_config`, associa a ela cardápio, pedidos, pagamentos e a equipe (clientes continuam globais), cria o plano interno **legado** (sem cobrança e sem limites) com uma assinatura ativa, grava o retrato do endereço nos pedidos de entrega antigos e define a loja inicial como **loja padrão**.
3. Crie o superadmin e revise a loja: nome, endereço, telefone, horários, fuso, zonas e taxas de entrega, prazo de retirada, credenciais Mercado Pago, logo e cor. Produtos sem adicionais marcados deixaram de aceitar adicionais.

Reverter

Prefira restaurar o backup e reverta sempre junto com o código anterior. `AuthSessionRefresh` tira o refresh das sessões em cookie (quem estava nelas entra de novo); `AccountEmailVerification` perde as confirmações e os links pendentes; `PerformanceIndexes` remove os índices (não a extensão); `StoreSlugHistoryAndCancelOrigin` perde a origem dos agendamentos e as reservas de slug; `PlanStorageLimit` e `AddressZoneLocation` removem as colunas; `AuthSessions` encerra as sessões de todas as contas e apaga a tabela de sessões; `OrderNumbersAndCustomerBlocks` remove os números dos pedidos e os bloqueios de clientes (que ficam só na auditoria; com bloqueios gravados, exige `DB_ALLOW_DESTRUCTIVE_REVERT=true`); `PlatformIntegrity` perde a origem das suspensões; `MaintenanceJobs` apaga o histórico da manutenção; `TenancyIndexes` remove os índices; `MultiTenantFoundation` só reverte com uma loja e recusa se for apagar auditoria, cobrança SaaS, sessões de suporte, convites ou redefinições, ou desfazer bloqueios e revogações de sessão, salvo com `DB_ALLOW_DESTRUCTIVE_REVERT=true`; `BaselineSchema` é irreversível.

Modelo de dados

Tabelas	Conteúdo
`stores`, `store_slug_history`	Loja: slug, dados comerciais, `account_status` (ACTIVE/SUSPENDED/INACTIVE), operação, entrega (zonas por bairro e/ou faixa de CEP, política fora das zonas), horários, fuso, credenciais MP cifradas, responsável. Slugs antigos reservados.
`plans`, `subscriptions`, `subscription_events`, `subscription_invoices`, `billing_webhook_events`	Planos (com `max_storage_mb`), assinaturas (com `suspended_by` e `cancel_scheduled_by`), histórico, faturas SaaS e notificações do provedor.
`platform_settings`, `audit_logs`, `support_sessions`, `maintenance_jobs`	Configurações da plataforma, auditoria, sessões de suporte e manutenção (otimização de imagens e limpeza de arquivos).
`auth_sessions`	Uma linha por token de sessão (`jti` único, conta, tipo `USER` / `SUPPORT`, validade, revogação, IP e navegador; transporte `BEARER` / `COOKIE` e o hash do refresh rotativo). Base do logout, das "Sessões ativas" e da sessão do site em cookie.
`users`, `account_email_tokens`, `addresses`, `invitations`, `password_reset_tokens`	Contas (equipe com `store_id`; clientes e superadmin sem loja; `email_verified_at`), links de conta, endereços (com bairro, cidade e UF do CEP), convites e redefinições.
`products`, `addons`, `orders`, `payments` (+ `product_allowed_addons`, `order_items`, `order_item_addons`)	Dados por loja (`store_id NOT NULL`, imutável). `orders.number`: número do pedido na loja (1..N), atribuído pelo banco e imutável.
`store_customer_blocks`, `store_order_counters`	Clientes bloqueados por loja (motivo, quem bloqueou, quando) e o contador do número do pedido por loja (usado só pelo gatilho).
`store_config`	Histórico da versão de loja única (não é mais usada).

---

7. Superadministrador e contas de demonstração

Comando `platform:superadmin`

O papel global nunca é concedido por HTTP. No servidor, com acesso ao banco:

```
cd backend
npm run platform:superadmin -- --email=ops@suaempresa.com.br --name="Nome Sobrenome"
npm run platform:superadmin:prod -- --email=ops@suaempresa.com.br --name="Nome Sobrenome"   # produção, após o build
```

Situação / opção	O que acontece
E-mail sem conta	Cria o superadmin com senha forte aleatória exibida uma única vez (ou `SUPERADMIN_PASSWORD`: 12+ caracteres, maiúscula, minúscula e número ou símbolo).

Situação / opção	O que acontece
Cliente já existente	Recusado, mostrando os dados da conta. `--promote-existing` promove, troca a senha, encerra as sessões e invalida os links de redefinição pendentes (o cadastro é público: alguém poderia ter "plantado" a conta). Prefira um e-mail sem conta.
Equipe de uma loja	Nunca vira superadmin: use outro e-mail.
`--reset-password`	Recuperação de acesso: senha nova e sessões encerradas (substitui o antigo `npm run seed -- --force`).
`--revoke`	Remove o papel global e encerra as sessões.

Tudo fica na auditoria. Superadmins criados pelo comando nascem com o e-mail confirmado. As demais contas recuperam o acesso por "Esqueci minha senha" ou por um link de redefinição de uso único gerado no superpainel (Usuários): com SMTP, esse link vai só para o e-mail da conta (o operador não o vê; se o envio falhar, nenhum link é gerado); sem SMTP, aparece uma vez para o operador. Contas da plataforma não recebem link pelo superpainel.

Seeds (somente development e test)

O quê	Quando
Loja demo **Lanchonete Demo** (`/loja/demo`), aberta, plano interno gratuito sem limites	Subida da API num banco **sem lojas**; vira a loja padrão (`SEED_DEMO_USERS`).
Equipe demo (administrador, atendente, cozinha, entregador) e cliente demo	Subida em development/test, se faltarem (`SEED_DEMO_USERS`). A equipe fica na loja padrão.
Superadmin demo (`superadmin@comestra.com.br`)	Em test por padrão; em development só com `SEED_DEMO_SUPERADMIN=true` ou `npm run seed -- --with-superadmin`.

- Na subida, só as contas que faltam são criadas; contas existentes nunca mudam. As contas demo nascem com o e-mail confirmado.
- `npm run seed` recria, tira da lixeira e redefine nome, perfil e senha das contas demo. Recusa produção, com ou sem `--force`.
- Em produção nada de demonstração é criado; `SEED_DEMO_USERS=true` gera erro no log.

Senhas públicas. Qualquer pessoa que alcance a API (rede local, `npm run tunnel` do app, ngrok) entra como ADMIN da loja padrão e, se existir, como superadmin de todas as lojas. Bancos de versões anteriores podem ter `superadmin@comestra.com.br` com a senha `Superadmin@123` (a subida avisa no log). Antes de expor a API: `npm run platform:superadmin -- --email=superadmin@comestra.com.br --reset-password` (ou `--revoke`), troque ou exclua as contas demo e use `SEED_DEMO_USERS=false`.

8. Isolamento entre lojas

Contexto	De onde vem a loja	Guard
Painel da equipe	`users.store_id` do usuário autenticado (recarregado do banco a cada requisição)	`@StaffRoute` / `@CurrentTenant()`
Vitrine e compra	`:storeSlug` da rota; sem slug: loja da equipe ou loja padrão	`@PublicStoreRoute` / `@PublicStore()`
Superpainel	Nenhuma loja implícita; a loja-alvo vem do id na rota	`@PlatformRoute()`

- Nenhum `storeId` /slug vindo de cabeçalho, corpo ou query concede acesso; os DTOs recusam campos extras.
- Recurso de outra loja (ou inexistente) → **404 uniforme**. Rota da equipe com slug de outra loja → 403.
- Todos os serviços usam o `storeId` validado em todas as consultas, inclusive alterações e exclusões.
- No banco: `store_id NOT NULL` + FKs, triggers de mesma loja (adicionais, itens, entregador, pagamento do pedido, eventos e faturas da assinatura, responsável, convites), `store_id` imutável, equipe sem troca de loja e o TypeORM configurado para lançar erro em filtros `undefined`.

Situação da loja × capacidades

Situação	Vende	Lê	Opera pedidos	Altera	Vitrine
OPERATIONAL (TRIALING/ACTIVE)	sim	sim	sim	sim	sim
PAST_DUE (na tolerância)	sim (aviso)	sim	sim	sim	sim
Loja SUSPENDED, assinatura suspensa ou cancelada, sem assinatura	não	sim	concluir, cancelar, estornar	não	indisponível
INACTIVE (desativada)	não	não	não	não	404

- O **cliente** que tenta comprar numa loja que não vende recebe sempre 403 "Esta loja está temporariamente indisponível para novos pedidos." O motivo interno aparece só para a equipe.
- Assinatura suspensa pela plataforma (`suspended_by = PLATFORM`): mensagem neutra para a equipe ("fale com o suporte"); pela inadimplência (`BILLING`): "suspensa por falta de pagamento".
- Equipe de loja desativada não entra no painel nem lê pelas rotas antigas (403), e as sessões e os links de redefinição pendentes dela são encerrados na desativação (restaurar a loja não traz os links de volta).
- Loja apenas fechada não é bloqueio: o pedido novo é recusado, mas um pagamento online já aprovado vira pedido.

9. Superpainel

Rotas `/platform/**` da API e telas em `/platform` no web; somente o token normal do superadmin (o de sessão de suporte e qualquer perfil de loja recebem 403). Toda ação é auditada.

Área	O que faz
Dashboard	Lojas por situação, assinaturas, MRR, ARR (projeção), churn de 6 meses, pedidos, GMV das lojas e receita da plataforma (blocos separados), distribuição por plano e alertas: `PAST_DUE_GRACE_ENDING`, `SUBSCRIPTION_SUSPENDED` (com `suspended_by`), `REFUND_PENDING`, `INVOICE_FAILED`, `NO_ACTIVE_ADMIN`, `STORE_SUSPENDED`, `TRIAL_ENDING`, `ONLINE_PAYMENTS_WITHOUT_CREDENTIALS`, `INACTIVE_STORE_BILLED` e `CANCELED_STILL_BILLED` (crítico: assinatura cancelada ainda autorizada ou cobrada no Mercado Pago, com o valor a devolver). Números com cache de até 30 s (`PLATFORM_INSIGHTS_CACHE_MS`).
Lojas	Lista com busca e filtros; criação (loja fechada, entrega desligada, assinatura e convite do responsável: link exibido uma vez sem SMTP, ou enviado só por e-mail com SMTP); detalhe sem segredos (com uso de armazenamento); slug (o antigo fica reservado para sempre contra o autoatendimento; renomear a loja padrão pede atualizar `DEFAULT_STORE_SLUG`), domínio e responsável; convites seguintes pelo detalhe (com SMTP, o link vai só por e-mail); suspender, reativar, desativar e restaurar (motivo obrigatório para suspender e desativar; transição inválida → 409; nada é apagado); equipe, convites, métricas e auditoria da loja.
Sessão de suporte	Motivo de 10 a 300 caracteres; o token vale como ADMIN da loja, **somente leitura em toda a API**, até expirar (padrão 30 min, configurável de 5 a 240) ou ser encerrado.
Usuários	Busca global, detalhe com eventos de autenticação (inclusive troca e confirmação de e-mail e reuso de sessão), bloquear (motivo), desbloquear, encerrar sessões e link de redefinição (válido 60 min; sem SMTP, exibido uma vez; com SMTP, enviado só ao dono). A lista e o detalhe de contas e a equipe de uma loja ficam na auditoria (leituras de dados pessoais).
Planos	Código, nome, preço, periodicidade, teste grátis, limites (equipe, produtos, pedidos do mês e **armazenamento em MB**; vazio ou "Ilimitado" = sem limite), recursos (pagamentos online, relatórios, identidade visual), ativo e público.
Assinaturas e faturas	Troca de plano (downgrade recusado se o uso, inclusive o armazenamento, não couber), cancelamento (motivo obrigatório no imediato; o agendado pela plataforma só ela desfaz), desfazer cancelamento agendado, suspender (também uma assinatura já suspensa por inadimplência) e reativar (409 em transição inválida); medidores de uso com armazenamento; faturas: registrar pagamento (idempotente), recusar e anular (motivo obrigatório).
Financeiro	Receita SaaS, faturas por status, MRR/ARR, GMV, top 10 lojas por GMV e inadimplência.
Auditoria	Filtros por loja, ator, resultado, escopo, período, ação (nome exato ou grupo) e alvo.
Configurações	Loja padrão, tolerância da inadimplência (0–60 dias, padrão 7), duração do suporte, autoatendimento de lojas e o plano dele.
E-mail transacional	Driver (`log` ou `smtp`), servidor SMTP (host:porta, TLS, autenticação), remetente e problemas da configuração, sem usuário nem senha. "Enviar e-mail de teste" (5 a cada 10 min por operador, auditado): enviado, driver de log ou o erro com a dica de correção.
Manutenção	Em segundo plano, uma execução por vez entre todas as instâncias: **otimizar imagens**; **limpar arquivos sem uso** em `/uploads` ("Verificar" só conta; "Apagar sem uso" pede confirmação e apaga só imagens sem referência há mais de 24 h, inclusive considerando a lixeira, os avatares e os logos; com `UPLOADS_CLEANUP_INTERVAL_HOURS`, a limpeza também roda sozinha, e a linha "Limpeza automática ligada" mostra a última e a próxima execução); **credenciais do Mercado Pago**: situação por formato e "Regravar credenciais" (antigo → atual, idempotente, sem expor tokens). O resumo e a auditoria guardam só números.

Metodologia. MRR = soma do preço mensalizado das assinaturas ACTIVE e PAST_DUE de planos pagos (a de uma loja desativada continua contando, e o alerta `INACTIVE_STORE_BILLED` pede cancelar ou restaurar). ARR = MRR × 12 (projeção). Churn = canceladas no mês ÷ pagantes no início do mês. A receita da plataforma (faturas SaaS pagas) nunca é somada ao GMV das lojas.

---

10. Planos e assinaturas

```
TRIALING  → ACTIVE | PAST_DUE | CANCELED
ACTIVE    → PAST_DUE | SUSPENDED | CANCELED
PAST_DUE  → ACTIVE | SUSPENDED | CANCELED
SUSPENDED → ACTIVE | CANCELED
CANCELED  → (final; nova contratação cria outra assinatura)
```

- **Rotina** (a cada `SAAS_BILLING_INTERVAL_MS`): fim do teste → ACTIVE (plano gratuito) ou fatura + PAST_DUE; fim do período → renovação (fatura) ou CANCELED (cancelamento agendado); fim da tolerância → SUSPENDED com `suspended_by = BILLING`.
- **Pagamento** (manual ou webhook validado) → fatura PAID + ACTIVE com período novo. Notificações repetidas não duplicam efeitos.
- **Suspensão pela plataforma** (`PLATFORM`): o pagamento quita a fatura mas não reativa; só a reativação pelo superadmin devolve a loja. O checkout da loja responde 409 e a página Assinatura mostra o aviso, sem o botão de pagamento.
- **Limites**: equipe ativa e produtos fora da lixeira (rígidos: trava por loja, inclusive na restauração e no aceite de convite), pedidos do mês (calendário da loja; conta também os pagamentos aprovados sem pedido, os cartões em análise e os PIX pendentes do próprio cliente) e **armazenamento** (imagens da loja, conferido antes de gravar: 403 com o aviso do plano; avatares não contam). Downgrade só se o uso couber. Recursos: pagamentos online, relatórios avançados, identidade visual.
- **Quem agendou o cancelamento** (`cancel_scheduled_by`): se foi a plataforma, a loja não desfaz (vê "fale com o suporte"). O cancelamento imediato pela loja (sem período pago a terminar) fica só com o responsável.
- Troca de plano vale na hora, sem proporcionalidade. Cancelamento imediato anula só faturas de períodos futuros. Uma fatura válida por período. `monthly_price` é arredondado em centavos.
- **Provedores**: `MANUAL` (faturas registradas pelo superadmin; o webhook SaaS sem token responde 200 e registra `IGNORED`) e `MERCADO_PAGO` (preapproval na conta da Comestra; implementado, ainda não validado contra o Mercado Pago real). Com o Mercado Pago: cobrança depois do cancelamento tenta cancelar a autorização de novo; preapproval órfã é cancelada; a rotina repete o cancelamento no provedor com intervalo crescente; o checkout é serializado por loja (409 se outro checkout da loja demorar; limite `SUBSCRIPTION_CHECKOUT`).

11. Painel da loja

Tela	Perfis	O que faz
`/dashboard`	`ADMIN`	Faturamento, ticket médio, fila de produção, gráficos e relatórios (conforme o plano), no fuso da loja.
`/orders`	Equipe	Quadro por etapa (a cada 5 s, pausado com a aba oculta), ações e visão por perfil, despacho e interruptor da loja (ADMIN e EMPLOYEE). Cartões com `Pedido #<número da loja>`. Pagamento estornado ou contestado no Mercado Pago depois da aprovação: "Estornado no Mercado Pago" em vermelho, alerta no cartão e faixa no topo (não entregue sem receber).
`/products`	`ADMIN`	Produtos, adicionais, disponibilidade, fotos e lixeira (respeita o limite do plano).
`/users`	`ADMIN`	Equipe da própria loja e convites (link de uso único; no máximo 3 por destinatário e 100 por loja em 24 h); respeita `max_staff_users` (limite rígido). O convite de troca do responsável, criado pela plataforma, aparece como "Criado pela plataforma" e não pode ser revogado pela loja.
`/customers`	`ADMIN` , `EMPLOYEE`	Clientes que compraram na loja e o histórico de pedidos dela; selo "Bloqueado" e filtro Todos/Bloqueados. Bloquear (motivo obrigatório) e desbloquear: só `ADMIN` .
`/trash`	`ADMIN`	Lixeira de produtos e da equipe.
`/settings`	`ADMIN`	Dados comerciais, endereço, horários, fuso, entrega e retirada (zonas por bairro — com os nomes dos Correios — e/ou faixa de CEP, taxas, e "Entregar com uma taxa padrão" ou "Recusar a entrega" fora das zonas), mensagens, logo e cor (se o plano permitir) e credenciais Mercado Pago (token só de escrita).
`/subscription`	`ADMIN`	Plano, limites e uso (com o medidor "Armazenamento"), faturas, pagamento (com provedor recorrente) e cancelamento no fim do período.

- **Responsável pela loja** (`owner_user_id`): outro ADMIN não troca e-mail, senha nem perfil dele, nem o exclui (403). Nome e telefone continuam editáveis. A troca do responsável é feita pelo superpainel.
- **Convites**: um em aberto por loja + e-mail; o convite só vale enquanto quem convidou continua ADMIN ativo da loja (ou superadmin ativo). Rebaixar, mandar para a lixeira ou bloquear um ADMIN cancela os convites dele.
- Regra do último administrador por loja; contas de clientes e da plataforma não são geridas pelo ADMIN.
- Loja nova: entrega desligada até configurar as zonas; sem credenciais Mercado Pago, só pagamento na entrega.
- **Cliente bloqueado**: não orça, não pede e não paga naquela loja (403 genérico "Não é possível fazer pedidos nesta loja. Fale com a loja.", sem o motivo) e continua comprando nas outras. Um PIX gerado antes do bloqueio e pago depois ainda vira pedido (cancele e estorne no quadro). Bloqueio e desbloqueio ficam na auditoria.

- **Sessões ativas** e **Minha conta** no menu lateral: aparelhos conectados, "Encerrar" e "Sair de todos os aparelhos"; e-mail confirmado, troca de e-mail e de senha (ocultos durante uma sessão de suporte).
- **Celular**: abaixo de 640 px, as listas viram cartões (rótulo: valor e ações no rodapé).

---

12. Vitrine e conta do cliente

Rota	Descrição
`/`	Diretório de lojas (busca).
`/loja/:slug`	Vitrine: cardápio, busca, categorias, combos, ofertas e banner do dia, com a identidade da loja.
`/loja/:slug/cart`, `/loja/:slug/order-success`	Carrinho, checkout e confirmação. `/cart` e `/order-success` antigos levam à última loja visitada.
`/signin`, `/signup`, `/esqueci-senha`, `/redefinir-senha/:token`	Conta do cliente e recuperação de senha por link de uso único.
`/my-orders`, `/profile`	Meus Pedidos (todas as lojas, os 100 mais recentes, com o número do pedido na loja; "Estornado" quando o pagamento foi desfeito no Mercado Pago) e Meus Dados (com "Sessões ativas" e "Segurança da conta": e-mail confirmado, troca de e-mail com confirmação e troca de senha com a senha atual).
`/verificar-email/:token`, `/confirmar-email/:token`, `/desfazer-troca-de-email/:token`	Links de conta: confirmação do e-mail (48 h), confirmação da troca no endereço novo (24 h; encerra todas as sessões) e "Não fui eu" no endereço antigo (7 dias; desfaz a troca com uma senha nova).
`/convite/:token`	Aceite de convite para a equipe de uma loja.
`/cadastrar-loja`	Autoatendimento de novas lojas (quando habilitado no superpainel).

- Carrinho guardado por loja (`comestra.cart.<storeId>`); PIX em andamento retomado ao voltar ao carrinho.
- Status da loja consultado a cada 30 s e ao voltar para a aba: com a loja fechada ou indisponível, o cardápio continua visível e o checkout fica bloqueado.
- Endereços: até 20 ativos por conta, busca de CEP (ViaCEP → BrasilAPI). A API confere o bairro pelo CEP e usa o bairro do CEP na taxa ("Pelo CEP, o bairro é …" quando difere do digitado). Numa loja que recusa endereços fora das zonas, o carrinho avisa junto do endereço ("Nada foi cobrado."), bloqueia o pagamento e oferece "Retirar no balcão". Troco: até R$ 200 acima do total.
- Sessão do site: token curto só na memória da aba e refresh num cookie `HttpOnly`; recarregar a página mantém a sessão; na vitrine, uma sessão vencida só deixa a aba sem login, sem aviso.
- Pagamento com cartão usa a chave pública da loja. Contas internas (equipe da loja e superadmin) veem o carrinho em "Modo Administrativo": sem formulário de cartão nem SDK do Mercado Pago, e com PIX e pagamento na entrega desabilitados (a API também recusa pedidos dessas contas).

13. Pedidos: regras, visão por perfil e transições

Preço calculado no servidor

```
POST /stores/demo/orders          (ou POST /orders, na loja padrão)
{
  "type": "DELIVERY",
  "addressId": 3,
  "items": [
    { "productId": 1, "quantity": 2, "addonIds": [4, 4], "meatPoint": "Ao Ponto",
      "removedIngredients": ["Cebola"], "observation": "Capricha no molho" }
  ],
  "paymentMethod": "MONEY",
  "changeFor": "R$ 100,00"
}
```

- O cliente nunca envia preços. Promoção só quando menor que o preço normal; cálculos em centavos; `addonIds` repetidos indicam a quantidade.
- Só adicionais permitidos no produto; **produto sem adicionais configurados não aceita nenhum** (400). Produto ou adicional indisponível, na lixeira ou de outra loja → "não encontrado", sem revelar o nome.
- Limites: 50 linhas, 50 unidades e 50 adicionais por item, observação de até 300 caracteres. Troco só em `MONEY`, maior ou igual ao total e no máximo R$ 200,00 acima.
- Retrato do preço (`unit_price`, `addons_snapshot`) e do endereço: o histórico não muda quando o cardápio ou o endereço mudam.
- Abuso: `ORDER_CREATE` (cliente + loja) e `ORDER_CREATE_IP`; no máximo N pedidos pagos na entrega aguardando a loja por cliente (N = limite de `ORDER_CREATE`, padrão 10) → 429.
- Entrega: a loja padrão usa as zonas da plataforma (`DELIVERY_ZONES_*`); as demais usam as zonas configuradas no painel e, sem elas, não entregam. Vale a primeira zona com o bairro **do CEP** (conferido pela API; sem o serviço, o digitado) ou com o CEP numa faixa. Fora das zonas: taxa padrão (`fallback`) ou recusa (`refuse` → 400 `DELIVERY_OUT_OF_AREA` no orçamento, no pedido e nas cobranças, antes de cobrar; pagamento já aprovado nunca é recusado). Retirada é grátis.
- **Número do pedido**: toda resposta de pedido traz `number`, o número do pedido **na loja** (1..N, sem buraco nem repetição, inclusive com pedidos simultâneos), atribuído pelo banco em todo pedido novo e imutável. As telas mostram `Pedido #<number>`; o `id` continua sendo a chave das rotas e do app mobile. `number` no corpo do pedido → 400.
- **Cliente bloqueado pela loja**: 403 no orçamento, no pedido pago na entrega e nas cobranças PIX/cartão daquela loja (rotas por slug e antigas). Não afeta outras lojas nem um pagamento já aprovado, que continua virando pedido.

O que cada perfil enxerga

Perfil	Pedidos	Dados do cliente
`ADMIN`, `EMPLOYEE`	Todos os pedidos da loja	Completos.
`KITCHEN`	Só `PENDING` e `PREPARING`	Só id e nome (sem endereço, troco nem pagamento).

Perfil	Pedidos	Dados do cliente
`COURIER`	Entregas prontas sem entregador; as próprias em andamento; as próprias concluídas ou canceladas nas últimas 24 h	Nome, telefone, endereço, total, troco e pagamento (nunca o e-mail).
`CLIENT`	Os próprios pedidos, em todas as lojas	Completos.

Fora da etapa → 404 uniforme, também nas alterações: cozinha e entregador recebem 404 "Pedido não encontrado." ao alterar ou cancelar um pedido fora da própria etapa (inclusive o entregador que perde a corrida por uma entrega). Listas: `GET /orders` e `/orders/my-orders` aceitam `?page` (1–10000) e `?pageSize` (1–100), com os 100 mais recentes por padrão; `scope=active` e os quadros de cozinha e entregador trazem até 500.

Transições de status por perfil

Ação	Gestão	Cozinha	Entregador
Aceitar: `PENDING → PREPARING`	sim	sim	—
Pronto: `PREPARING → READY_FOR_PICKUP`	sim	sim	—
Despachar com entregador escolhido (`/assign`)	sim	—	—
Pegar para entrega: `READY_FOR_PICKUP → DELIVERING`	sim	—	sim (assume se sem responsável)
Confirmar entrega: `DELIVERING → DONE`	sim	—	só as próprias
Balcão (retirada): `READY_FOR_PICKUP → DONE`	sim	—	—
Cancelar	até concluir	—	—

- `DONE` e `CANCELED` são finais; `TAKEOUT` nunca entra em `DELIVERING`. Transição impossível → 400; não permitida ao perfil → 403; pedido alterado por outra pessoa → 409.
- O cliente cancela enquanto `PENDING`. Pedido pago online cancelado gera estorno (`REFUNDED` , ou `REFUND_PENDING` se falhar).
- Despacho: só `DELIVERY` em `PREPARING` , `READY_FOR_PICKUP` ou `DELIVERING` , com um `COURIER` ativo da loja.
- Pedido pago na entrega vira `PAID` ao ser concluído. O faturamento conta só pedidos pagos e não cancelados.

---

14. Pagamentos do delivery

Por loja

- Cada loja informa a própria conta Mercado Pago em Configurações (token cifrado com `CREDENTIALS_ENCRYPTION_KEY` , vinculado à loja, nunca devolvido). O `MP_ACCESS_TOKEN` do ambiente vale só para a loja padrão sem credencial própria. Loja sem credencial: só pagamento na entrega.
- A chave pública da loja sai no perfil público (tokenização no navegador). Idempotência e estorno por loja.

Como o modo é decidido

	Sandbox sem `MP_ACCESS_TOKEN`	**Sandbox com token `TEST-`**	**Produção**
Cartão	Aprovação simulada.	Enviado ao Mercado Pago. Recusa ou falha → aprovação simulada.	Resultado real.
PIX	QR de teste, aprovado após `PIX_SANDBOX_APPROVAL_SECONDS`.	QR do Mercado Pago, aprovado após o prazo. Falha → QR de teste.	Aprovado só quando o Mercado Pago confirma.

Sandbox só com `NODE_ENV=development` ou `test`. Cartão de teste: `5031 4332 1540 6351`, validade `11/30`, CVV `123`, titular `APRO`, CPF `123.456.789-09`.

Fluxo

1. O front envia o pedido e uma `idempotencyKey` para `/stores/:slug/payment/card` ou `/pix`. A API calcula o valor, grava o pagamento (`payments`, com o retrato do carrinho) e cobra.
2. Aprovado o pagamento (resposta do cartão, `GET /payment/status/:id`, webhook ou conciliação a cada 60 s), **a própria API cria o pedido**, uma única vez.
3. Se a loja não puder mais vender (suspensa, desativada, assinatura suspensa/cancelada ou ausente), o pedido não é criado: o valor é estornado e o caso fica na auditoria. Loja apenas fechada continua recebendo o pedido.

- Limites: `PAYMENT_CREATE` (cliente + loja) e `PAYMENT_CREATE_IP`; até 5 cobranças pendentes por cliente e loja; o mesmo carrinho com um PIX pendente devolve o mesmo PIX.
- Webhook (`POST /payment/webhook`): corpo não confiável, reconsulta no Mercado Pago no máximo a cada 15 s por pagamento, limite por IP e fila limitada.
- Conciliação: 50 pagamentos por rodada, no máximo 10 por loja, aprovados sem pedido primeiro; cancela os pendentes de lojas que não vendem.
- `GET /payment/status/:id`: o pagador ou ADMIN/EMPLOYEE da loja do pagamento; consulta o Mercado Pago no máximo a cada 5 s por pagamento.
- **Estorno ou contestação feitos no Mercado Pago** depois da aprovação (com `MP_NOTIFICATION_URL`): o webhook reconsulta também os pagamentos aprovados, o pedido fica com `payment_status = REFUNDED` e a auditoria registra `payment.reversed`; o quadro avisa a equipe.
- Com várias instâncias, a criação do pedido de um pagamento aprovado usa uma trava do banco por pagamento: o pedido nunca nasce enquanto outra instância estorna.

15. Segurança

Sessões e contas

- JWT com versão de sessão (`token_version`): trocar ou redefinir a senha, confirmar ou desfazer uma troca de e-mail, bloquear, "encerrar sessões", desativar a loja (equipe) e promover pelo comando invalidam os tokens antigos.
- **Sessões registradas**: todo token emitido pela API leva `jti` e ganha uma linha em `auth_sessions` , gravada antes de a resposta sair (se a gravação falhar, a resposta é 500 e nenhum token sai). Token com `jti` só vale com a sessão existente, não revogada e no prazo; token antigo sem `jti` só vale na janela de `JWT_LEGACY_ISSUED_BEFORE` (em produção, sem ela, é recusado). No máximo 100 sessões ativas por conta (as mais antigas caem). Sair (`POST /auth/logout`) encerra a sessão no servidor; "Sessões ativas" lista os aparelhos e encerra um ou todos (`POST /auth/logout-all`). Sessões mortas são apagadas a cada sessão nova da conta e, no máximo uma vez por hora por instância, em lotes de 5000 vencidas.
- **Sessão do site em cookie**: com o cabeçalho `X-Auth-Transport: cookie` (só o site envia), o login devolve um token de 15 min, guardado só na memória da aba, e grava um refresh rotativo num cookie `HttpOnly` , `SameSite=Strict` (`__Host-comestra_rt` em produção ou HTTPS; `comestra_rt` em http fora de produção). `POST /auth/web/refresh` renova (o refresh anterior ainda vale por `AUTH_REFRESH_GRACE_SECONDS` ; reuso fora disso encerra a sessão inteira e é auditado como `auth.refresh.reuse`); `POST /auth/web/logout` encerra. Um login novo no mesmo navegador encerra a sessão anterior no servidor. O app mobile não envia o cabeçalho e não muda. Kill switches: `AUTH_WEB_COOKIE=false` (API) e `VITE_AUTH_TRANSPORT=bearer` (build).
- Conta bloqueada não entra. Recuperação de senha por link de uso único (hash no banco; no máximo 3 por conta em 24 h): com `MAIL_DRIVER=smtp` o link vai por e-mail; com o driver de log, o superadmin gera o link (com SMTP, o link do superpainel vai só ao dono). Links pendentes deixam de valer quando a conta troca e-mail ou senha, é bloqueada, vai para a lixeira, encerra as sessões ou a loja da equipe é desativada.
- **Minha conta** (modo `compat`): troca de e-mail só depois da confirmação no endereço novo (resposta igual para endereço livre, em uso ou reservado); o endereço antigo recebe "Não fui eu", que por 7 dias desfaz a troca com uma senha nova, e fica reservado nesse período; troca de senha com a senha atual, com aviso ao dono. A rota antiga do app (`PATCH /users/:id`) continua trocando o e-mail na hora.
- Em development sem `JWT_SECRET` , o segredo dos tokens é aleatório por instalação (`backend/.temp/dev-jwt-secret`), não mais derivado do nome da marca.
- Sessão de suporte: qualquer `POST` , `PUT` , `PATCH` ou `DELETE` com o token de suporte → 403 "Sessão de suporte é somente leitura: nenhuma alteração pode ser feita." Única exceção: `POST /auth/logout` , que encerra a sessão de suporte com o próprio token (auditado).
- Senha atual (`current_password`) obrigatória para contas da plataforma ao trocar e-mail ou senha e em Minha conta; opcional na rota antiga enquanto o app mobile não a envia. A regra de senha é validada em tempo linear (sem ReDoS), com as mesmas mensagens.
- Avatar em texto só dos ícones prontos; fotos enviadas levam o dono no nome e a anterior é apagada.

Limites de tentativas e de abuso

Política	Padrão	Onde
`LOGIN_FAILURES` / `LOGIN_FAILURES_IP`	10/900 · 100/900	Falhas de login por IP + e-mail e por IP (a vaga é reservada antes de conferir a senha).

Política	Padrão	Onde
`LOGIN_FAILURES_ACCOUNT` / `LOGIN_SUCCESS`	50/3600 · 200/600	Falhas por conta somando os IPs (acima do teto, só a senha certa vinda de um IP com sessão ativa entra; todo 429 do login demora o mesmo que uma conferência) e logins certos por IP + e-mail.
`FORGOT_PASSWORD` / `TOKEN_REDEEM`	5/900 · 20/900	"Esqueci a senha"; resgate de links de redefinição e de convite.
`SIGNUP` / `STORE_SIGNUP` / `CEP_LOOKUP`	30/3600 · 5/3600 · 60/60	Cadastro público (antes de ler o corpo), autoatendimento de lojas e busca de CEP.
`ORDER_CREATE` / `ORDER_CREATE_IP`	10/600 · 60/600	Pedidos (cliente + loja; também é o teto de pedidos pendentes pagos na entrega) e por IP.
`PAYMENT_CREATE` / `PAYMENT_CREATE_IP` / `PAYMENT_WEBHOOK`	10/600 · 60/600 · 300/60	Cobranças do delivery e webhook (por IP).
`UPLOAD` / `ADDRESS_WRITE`	30/600 · 30/600	Imagens (por conta ou IP) e endereços (por conta).
`TEAM_WRITE` / `CREDENTIALS_CHANGE`	30/600 · 10/900	Equipe e convites (por loja); falhas na troca das próprias credenciais (por conta e IP).
`ACCOUNT_EMAIL` / `ACCOUNT_EMAIL_TARGET`	5/3600 · 3/3600	Pedidos de confirmação e troca do próprio e-mail (por conta, 429) e e-mails por destinatário (silencioso).
`REFRESH` / `SUBSCRIPTION_CHECKOUT`	120/300 · 10/600	Renovações da sessão do site (por IP) e links de pagamento da assinatura (por loja).
`TEST_EMAIL`	5/600	E-mail de teste do superpainel, por operador.

Formato `limite/segundos` em `RATE_LIMIT_<NOME>`. Contadores em memória, por instância; as chaves por IP agrupam o IPv6 por /64. No banco (valem entre instâncias): "esqueci a senha" (3 links por conta em 24 h) e convites da loja (3 por destinatário na loja e 100 por loja em 24 h; a partir de 10 ao mesmo e-mail somando as lojas, o convite sai sem e-mail). Atrás de proxy ou túnel, configure `TRUST_PROXY` com o número de proxies.

Uploads

JPG, PNG ou WebP de até 5 MB (tipo conferido pelo conteúdo; só esses decodificadores ficam ligados no libvips), até 40 megapixels, multipart com até 300 campos de 16 KB, nomes de campo com no máximo 2 níveis e índice até 300 (400 nos demais), no máximo 2 conversões simultâneas; conversão para WebP (800 px, qualidade 80). Cota de armazenamento do plano conferida antes de gravar (403). Imagens da loja levam o prefixo `s<storeId>-`; a imagem substituída de produto ou adicional é apagada quando nenhum outro cadastro a usa.

Cabeçalhos HTTP da API

Sem `X-Powered-By`. Em toda resposta: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, `Cross-Origin-Resource-Policy: cross-origin` e HSTS em produção. CSP: `default-src 'none'; frame-ancestors 'none'` na API; em `/uploads`, também `style-src 'unsafe-inline'` e `sandbox`; no Swagger, só `frame-ancestors 'none'`. Respostas comprimidas (gzip/br) acima de 1 KB; corpo JSON acima de 100 KB → 413 (2 MB em `/store/settings`). CORS com credenciais só para as origens do site (`AUTH_COOKIE_ORIGINS` ou `CORS_ORIGINS` + `APP_WEB_URL`).

SPA (frontend web)

- O build de produção injeta a Content-Security-Policy por `<meta>` (origem de `VITE_API_URL`, Mercado Pago, Iconify e o host dos avatares). O Vite de desenvolvimento não tem CSP.
- O servidor que hospeda o build deve enviar os cabeçalhos de `securityHeaders()` (`vite.config.ts`): CSP com `frame-ancestors 'none'`, `X-Frame-Options: DENY`, `nosniff`, `Referrer-Policy: strict-origin-when-cross-origin`, `Permissions-Policy` e HSTS em HTTPS.
- Sessão por aba, sincronizada entre abas (login, saída, troca de conta, suporte); chamadas da sessão anterior são canceladas. No modo cookie, o token fica só na memória e o `localStorage` guarda só o marcador `comestra.session` (sem segredo); cada aba renova o próprio token (na subida, a 80% da validade, antes de chamadas e depois de um 401), com Web Lock entre abas. Sair apaga os carrinhos e a última loja (o tema fica) e chama `POST /auth/logout` (e `/auth/web/logout`) como melhor esforço, sem travar a saída.
- Suporte iniciado numa sessão em cookie: nada do superpainel fica na aba; encerrar o suporte volta ao superpainel pelo refresh, mesmo depois de recarregar. Numa sessão bearer antiga, o token do superpainel fica só na memória da aba que o iniciou. "Encerrar e voltar ao superpainel" e "Sair do Sistema" encerram o suporte com o próprio token de suporte, na hora, mesmo depois de recarregar a página.
- Com a CSP, o script antifraude do Mercado Pago só roda com um nonce gerado pelo servidor (o pagamento com cartão funciona sem ele).

Auditoria

Ator, escopo, loja, ação, alvo, resultado, IP, navegador e metadados sem senhas, tokens nem segredos: ações do superpainel, autenticação (inclusive `auth.logout` e `auth.session.revoke`), equipe e convites, configurações e credenciais, assinaturas, sessões de suporte (com `via` quando encerradas pelo próprio token), bloqueio e desbloqueio de clientes (`store.customer.block` / `unblock`), e-mail de teste, limpeza de arquivos (manual ou agendada, como sistema) e regravação de credenciais (só contagens), pagamentos que não viram pedido ou estornados no Mercado Pago (`payment.reversed`), os 409 de e-mail em uso, troca e confirmação de e-mail (`auth.email_change.*`, `auth.email_verification.*`), reuso de sessão (`auth.refresh.reuse`) e as leituras de dados pessoais no superpainel (`platform.users.list` / `view`, `platform.store.team.view`).

16. Mapa de endpoints da API

Base local: `http://localhost:3000`. Schemas no Swagger (`/api`); contratos e mensagens em `docs/saas/arquitetura.md`. Rotas `stores/:storeSlug/...` atuam na loja do slug; as antigas sem slug, na loja da equipe (leituras) ou na loja padrão.

Área	Rotas	Acesso
Saúde	`GET /health` → `{ "status": "ok" }` (sem nome nem versão)	Público
Autenticação	`POST /auth/login`, `GET /auth/me`, `POST /auth/logout`, `POST /auth/logout-all`, `GET /auth/sessions`, `DELETE /auth/sessions/:id`, `POST /auth/forgot-password`, `GET /auth/reset-password/:token`, `POST /auth/reset-password`, `POST /auth/web/refresh`, `POST /auth/web/logout`	Público (limitado) / autenticado
Minha conta	`GET /account/security`, `POST /account/password`, `POST/DELETE /account/email-change`, `POST /account/email-verification`; links: `GET /account/email-change/:token`, `POST /account/email-change/confirm`, `GET /account/email-revert/:token`, `POST /account/email-revert`, `GET /account/email-verification/:token`, `POST /account/email-verification/confirm`	Autenticado / público (limitado)
Convites	`GET /invitations/:token`, `POST /invitations/:token/accept`	Público (limitado)
Conta e equipe	`POST /users`, `GET /users`, `GET /users/trash`, `GET/PATCH/DELETE /users/:id`, `PATCH /users/:id/restore`, `DELETE /users/:id/permanent`	Próprio usuário · ADMIN (equipe da loja)
Painel da loja	`GET/PATCH /store/status`, `GET/PATCH /store/settings`, `PUT/DELETE /store/settings/payments`, `POST/DELETE /store/logo`, `GET/POST /store/invitations`, `DELETE /store/invitations/:id`, `GET /store/customers`, `GET /store/customers/:userId`, `POST/DELETE /store/customers/:userId/block`	ADMIN (status e leitura dos clientes: também EMPLOYEE)
Assinatura da loja	`GET /store/subscription`, `POST /store/subscription/{checkout,cancel,resume}`	ADMIN
Vitrine	`GET /stores`, `GET /stores/default`, `GET /stores/:storeSlug`, `GET /stores/:storeSlug/status`, `GET /plans`, `POST /stores/signup`	Público
Cardápio	`GET /products`, `GET /products/:id`, `GET /addons` (e por slug); `GET /products/trash`, `POST/PATCH/DELETE /products...`, `PATCH /products/:id/restore`, `DELETE /products/:id/permanent`, `POST/PATCH/DELETE /addons...`	Público · ADMIN
Pedidos	`POST /orders/quote`, `POST /orders`, `GET /orders/my-orders`, `PATCH /orders/:id/cancel`; `GET /orders`, `GET /orders/:id`, `PATCH /orders/:id`, `PATCH /orders/:id/assign`, `GET /orders/:id/nearby`, `GET /orders/drivers/list`, `GET /orders/summary`, `GET /orders/charts`, `GET /orders/reports`	Cliente · equipe (conforme o perfil)

Área	Rotas	Acesso
Pagamentos	`POST /payment/card`, `POST /payment/pix`, `GET /payment/status/:id`, `POST /payment/:id/cancel`, `POST /payment/webhook`	Cliente · webhook público
Endereços	`GET /addresses/cep/:cep`, `POST/GET /addresses`, `GET/PATCH/DELETE /addresses/:id`	Público (CEP) · dono
Banner e imagens	`GET /promotions/banner`, `POST /uploads`	Público · ADMIN
Superpainel	`/platform/dashboard`, `/platform/stores/**`, `/platform/users/**`, `/platform/auth-events`, `/platform/plans/**`, `/platform/subscriptions/**`, `/platform/invoices/**`, `/platform/finance`, `/platform/audit`, `/platform/settings`, `/platform/settings/test-email`, `/platform/support-sessions/**`, `/platform/maintenance/{optimize-images,cleanup-uploads,store-credentials,store-credentials/reencrypt}`	`SUPER_ADMIN`
Cobrança SaaS	`POST /billing/webhook/mercadopago`	Público (assinatura validada)

Listas novas: `{ items, total, page, pageSize }` (`pageSize` até 100). Ids fora de `1..2147483647` → 400. Lista enviada onde se espera objeto → 400.

---

17. API: comandos e comportamento global

Comando (em `backend/`)	Finalidade
`npm run start:dev` / `start:debug`	Desenvolvimento com recarga automática (e inspetor).
`npm run build` / `npm run start:prod`	Build em `dist/`; execução sempre com `NODE_ENV=production`.
`npm run migration:*`	Migrations (seção 6).
`npm run platform:superadmin` / `:prod`	Superadmin (seção 7).
`npm run seed`	Loja e contas de demonstração (só dev/test).
`npx jest` / `npm run test:cov` / `npm run test:e2e`	Testes unitários, cobertura e E2E da API com Jest.
`npm run build:e2e`	Build próprio da suíte Playwright (`dist-e2e/`).
`npm run lint` / `npm run format`	ESLint e Prettier.

- **Subida:** registra `Ambiente: <modo>` e avisos de configuração (inclusive do e-mail); aplica as migrations; cria a demo (dev/test); escuta em `HOST`:`PORT` e registra `API Comestra escutando em <host>:<porta> (<alcance>)` e `rodando em http://...`. Uma falha no seed não derruba a API; uma falha ao escutar registra o código do erro e encerra.
- **Validação:** `ValidationPipe` com `whitelist`, `forbidNonWhitelisted` e `transform` (`src/config/app-setup.ts`, a mesma configuração dos testes).

- **Usuário a cada requisição:** a `JwtStrategy` recarrega o usuário e, para tokens com `jti`, confere a sessão registrada (uma consulta indexada); conta excluída, bloqueada, com sessões encerradas ou sessão revogada/vencida → 401.
- **Rotinas:** ciclo das assinaturas (com novas tentativas de cancelar no Mercado Pago), conciliação dos pagamentos (60 s), heartbeat da manutenção, limpeza das sessões mortas e, se ligada, a limpeza agendada dos arquivos sem uso (`UPLOADS_CLEANUP_INTERVAL_HOURS`).
- **Banco:** pool de `DB_POOL_SIZE` conexões (padrão 10), cada consulta da API com até `DB_STATEMENT_TIMEOUT_MS` (60 s; as migrations não têm esse limite) e espera máxima por conexão de `DB_POOL_ACQUIRE_TIMEOUT_MS` (30 s). O quadro de pedidos e o dashboard do superpainel usam caches de 30 s.
- **URLs:** `/health` (monitoramento; só `{"status":"ok"}`), `/api` (Swagger, dev/test), `/uploads/<arquivo>`.

E-mail transacional (SMTP)

Convites, links de redefinição de senha, os e-mails de conta (confirmação, troca de e-mail, "Não fui eu", senha alterada) e o e-mail de teste saem pelo driver de `MAIL_DRIVER`, em **texto e HTML**: convite e redefinição da equipe com o nome, a cor e o logo da loja (logo só com `PUBLIC_API_URL`); o resto com a marca da plataforma. Com `log` (padrão) nada é enviado: em dev/test o texto vai para o log; em produção só destinatário e assunto, com aviso na subida. Com `smtp`, a API envia pelo servidor configurado (nodemailer):

```
MAIL_DRIVER=smtp
SMTP_HOST=smtp.seuprovedor.com.br
SMTP_PORT=587            # STARTTLS; ou 465 com SMTP_SECURE=true
SMTP_SECURE=false
SMTP_USER=usuario-do-smtp
SMTP_PASSWORD=senha-do-smtp
MAIL_FROM="Comestra <nao-responda@seudominio.com.br>"
```

- Teste em Superpainel → Configurações → **E-mail transacional** → "Enviar e-mail de teste". Erros vêm com a dica: `EAUTH` → usuário/senha; `EDNS` → `SMTP_HOST`; `ETLS` / `ECONNECTION` → `SMTP_PORT` e `SMTP_SECURE`; `ETIMEDOUT` → host, porta e firewall; `EENVELOPE` → `MAIL_FROM` e destinatário.
- Com usuário e senha e `SMTP_SECURE` desligado, o STARTTLS é obrigatório. Prazos: 10 s para conectar, 10 s para a saudação e 30 s no socket. Em desenvolvimento, um Mailpit/MailHog local (porta 1025, sem autenticação) funciona com `SMTP_SECURE=false`.
- Falha no envio: uma linha no log (sem usuário nem senha) e o fluxo continua: o convite da loja é criado e o link aparece no painel; "esqueci a senha" continua respondendo 202. A criação de convite espera o envio SMTP; o "esqueci a senha", não. No superpainel com SMTP (redefinição e convites do detalhe da loja), a falha não gera link (503).
- Com `MAIL_DRIVER=log` em produção, a troca de e-mail pelo site não se completa (o link não chega a ninguém). O driver de log mostra só o texto: para ver o HTML em desenvolvimento, use o Mailpit/MailHog. Os logos são WebP (o Outlook clássico não os exibe).

18. Frontend web: comandos, rotas e sessão

```
cd frontend-web
npm install
npm run dev        # http://localhost:5173
npm run build      # verificação de tipos + build em dist/ (com a CSP)
npm run preview    # serve o build com os cabeçalhos de segurança
npm run lint
```

Área	Rotas	Guarda
Plataforma (pública)	`/`, `/cadastrar-loja`	—
Vitrine	`/loja/:slug`, `/loja/:slug/cart`, `/loja/:slug/order-success`	—
Conta	`/signin`, `/signup`, `/my-orders`, `/profile`, `/esqueci-senha`, `/redefinir-senha/:token`, `/convite/:token`, `/verificar-email/:token`, `/confirmar-email/:token`, `/desfazer-troca-de-email/:token`	`RequireAuth` (Meus Pedidos, Meus Dados)
Painel da loja	`/login`, `/dashboard`, `/orders`, `/products`, `/users`, `/trash`, `/customers`, `/settings`, `/subscription`	`StaffRoute` + `RoleRoute`
Superpainel	`/platform` e `/platform/{stores,stores/new,stores/:id,plans,subscriptions,users,finance,audit,settings}`	`PlatformRoute`

- Kit de UI em `src/components/ui`: `PageHeader`, `DataTable`, `Pagination`, `SearchInput`, `FilterTabs`, `StatusBadge`, `EmptyState`, `ErrorState`, `Modal`, `FormField`, `StatCard`, `Tabs`, `CopyField`, `Switch`. O `DataTable` com `stackOnMobile` vira cartões abaixo de 640 px.
- Sessão: no modo cookie (padrão, `VITE_AUTH_TRANSPORT=auto`), token curto só em memória, refresh em cookie `HttpOnly` e o marcador `comestra.session`; no modo bearer (sessões antigas, suporte, site e API em sites diferentes, ou `VITE_AUTH_TRANSPORT=bearer`), `comestra.token` como antes. `comestra.user` é conferido em `GET /auth/me`; cada aba usa o próprio token em memória. Expiração: `/login` no painel e no superpainel ("Sua sessão expirou"); na loja, a aba só fica sem login (as rotas protegidas levam a `/signin?redirect=...`).
- "Sessões ativas" (`components/Sidebar/ActiveSessions.tsx`) em Meus Dados e no menu lateral do painel e do superpainel; sair chama `POST /auth/logout` (melhor esforço, até 5 s).
- Pedidos exibidos como `Pedido #<número da loja>` no quadro, no modal do entregador, no Dashboard, em Meus Pedidos, na confirmação e no histórico do cliente (`orderNumber()` cai no id quando a resposta não traz `number`).
- Atualizações periódicas (pausadas com a aba oculta e retomadas na volta): status da vitrine a cada 30 s, quadro de pedidos a cada 5 s (sem redesenhar quando nada mudou), indicadores a cada 15 s, gráficos a cada 60 s, Meus Pedidos a cada 10 s. O Recharts fica num pedaço próprio do build (`charts`), carregado sob demanda.
- Servidor de produção: rewrite de todas as rotas para `index.html` e os cabeçalhos da seção 15.

19. App mobile (fora do escopo)

O app (`frontend-mobile/` , Expo SDK 54) não foi alterado nesta fase. Ele continua chamando as rotas antigas, que atuam na **loja padrão** (clientes e visitantes) ou na loja da equipe autenticada. Mudanças visíveis: a cozinha e o entregador só veem a própria etapa; listas antigas trazem no máximo 100 ou 500 itens; o ADMIN recebe 403 ao trocar e-mail, senha ou perfil do responsável da loja; novos 429 chegam com mensagem. Desta etapa: o app ainda mostra `Pedido #<id>` (o número da loja vem no campo `number`); o cliente bloqueado recebe o 403 com a mensagem genérica; a saída do app não chama `POST /auth/logout` (o token vale até expirar). Das levas seguintes (só campos aditivos e mensagens): o app nunca recebe a sessão em cookie (não envia o cabeçalho); um token antigo sem `jti` fora da janela de `JWT_LEGACY_ISSUED_BEFORE` leva o app de volta ao login; cozinha e entregador veem "Pedido não encontrado." numa corrida; uma loja padrão que recuse endereços fora da área devolve a mensagem do 400 no orçamento; a troca de e-mail pelo app continua imediata, com aviso e "Não fui eu" no endereço antigo.

```
cd frontend-mobile
npm install
npm run start:lan     # Expo em modo LAN (mesmo Wi-Fi)
npm run android       # emulador Android
npm run tunnel        # túnel para a API (grava EXPO_PUBLIC_API_URL no .env.local)
```

> O túnel expõe a API de desenvolvimento na internet. Com contas de demonstração (senhas públicas), qualquer pessoa entra como ADMIN da loja padrão. Veja a seção 7 antes de usar o túnel.

Pendências para a fase mobile: escolha de loja por slug, carrinho por loja, chave pública Mercado Pago por loja, recuperação de senha, envio de `current_password` e tratamento da sessão encerrada após a troca de senha. Detalhes no `frontend-mobile/README.md` .

20. Testes

Unitários da API (Jest)

```
cd backend
npx jest            # ou npm test
npm run test:cov    # cobertura
```

Sem banco. Cobrem isolamento (guards e situação da loja), superpainel, planos e assinaturas, auditoria, guardas das migrations, pedidos, pagamentos, autenticação, sessão em cookie, conta e e-mail, convites, limites, uploads, cota de armazenamento, frete por CEP, desempenho e configuração.

Web

```
cd frontend-web
npm run build       # tsc -b + vite build
npm run lint
```

Suíte ponta a ponta (Playwright)

```
cd e2e
npm install
npm test            # recria o banco comestra_e2e, compila a API (build:e2e) e roda tudo
npm run test:api    # só a API
npm run test:web    # só o navegador
npm run report      # relatório HTML
```

- Ambiente isolado: API de teste em `:3001` (banco `comestra_e2e`, schema só pelas migrations, sandbox ligado, cobrança SaaS `MANUAL`, e-mail sempre pelo driver de log, rotina das assinaturas a cada 10 s, limites de abuso afrouxados, tolerância do refresh de 3 s e superpainel sem cache) e web de teste em `:5174`. Sem `CEP_LOOKUP_FAKE`, 4 testes de frete por CEP são pulados; sem `UPLOADS_CLEANUP_INTERVAL_HOURS`, o da limpeza agendada.
- Suítes de API: `identity` (inclui `sessions.spec.ts`: logout, lista e encerramento de sessões, suporte encerrado pelo próprio token; e `maintenance.spec.ts`: limpeza de `/uploads` e regravação das credenciais), `commerce` (inclui `order-numbers.spec.ts`: número por loja, pedidos simultâneos sem buraco, imutabilidade), `tenancy` (isolamento entre lojas e `customer-blocks.spec.ts`), `platform` (superpainel, matriz de acesso, sessão de suporte e `mail.spec.ts`) e `billing` (planos, assinaturas, faturas, ciclo de vida, webhook, `storage.spec.ts` e `integrity.spec.ts` com as garantias do banco via SQL), mais `web-session.spec.ts` e os specs de conta em `identity`, `delivery-zones.spec.ts` e `performance.spec.ts` em `commerce`, `review2-stores.spec.ts` em `platform` e `security/request-parsing.spec.ts`. Web: `client` (inclui `security-headers.spec.ts`, `session-security.spec.ts`, `cookie-session.spec.ts` e `account-security.spec.ts`), `admin` (inclui `session-security.spec.ts` e `cookie-session.spec.ts`), `platform` (superpainel no navegador, inclusive no celular) e `multistore` (vitrine com várias lojas).
- Execuções paralelas: `E2E_DB_DATABASE` (terminado em `_e2e`), `E2E_API_PORT`, `E2E_WEB_PORT`, `E2E_UPLOADS_DIR`, `E2E_OUTPUT_DIR`, `E2E_REPORT_DIR` e `E2E_SKIP_BUILD=1`, com a API compilada uma vez antes. Exporte todas no mesmo comando: sem elas, o Playwright sobe servidores nas portas padrão com o banco `comestra_e2e`.

- O spec de CSP é pulado contra o Vite de desenvolvimento: sirva o build (`vite build` + `vite preview` na porta da web de teste, com `VITE_API_URL` da API de teste) e rode `npx playwright test tests/web/client/security-headers.spec.ts` (passo a passo em `e2e/README.md`).

Verificações recomendadas depois de instalar ou atualizar

Verificação	Resultado esperado
`npm run migration:show`	As 13 migrations com `[X]` .
`npm run migration:check`	"No changes in database schema were found".
`GET /health`	`{"status":"ok"}`
Log da subida	`Ambiente: production` (ou `development`), `escutando em <host>:<porta>` com o alcance esperado e nenhum aviso de configuração.
Superpainel → Configurações → E-mail transacional	Driver esperado e "Enviar e-mail de teste" com sucesso (com `smtp`).
Login do superadmin em `/login`	Abre `/platform` ; recarregar a página mantém a sessão (refresh em cookie: `POST /auth/web/refresh` responde 200).
`cd backend && npx jest` · `cd e2e && npm test`	Todas as suítes passando.

---

21. Produção

1. `NODE_ENV=production` , `JWT_SECRET` forte (`node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"`) e `CREDENTIALS_ENCRYPTION_KEY` (32 bytes) guardada em cofre.
2. `CORS_ORIGINS` com os domínios reais; `TRUST_PROXY` com o número de proxies (ex.: `1` ; nunca `true`); `SWAGGER_ENABLED` desligado.
3. `JWT_LEGACY_ISSUED_BEFORE` com o instante do deploy que aplica as sessões registradas (ou aceite que todos entrem de novo). Sessão do site em cookie: site e API no mesmo site (de preferência a mesma origem, com proxy em `/api`), HTTPS e o proxy sem registrar `Cookie` , `Set-Cookie` e `Authorization` nem os tokens dos links nos logs (o Nginx do Docker já faz isso).
4. `DB_SYNCHRONIZE=false` ; migrations aplicadas (`npm run migration:run:prod` , ou `DB_MIGRATIONS_RUN` com uma instância); backup antes de cada deploy com migrations.
5. Superadmin criado com `npm run platform:superadmin:prod` ; nenhuma conta de demonstração nem senha pública no banco.
6. Credenciais Mercado Pago de cada loja pelo painel, `PAYMENT_SANDBOX` desligado e `MP_NOTIFICATION_URL` pública em HTTPS.
7. HTTPS na frente da API e do site. Build da web com `VITE_API_URL` real, rewrite da SPA para `index.html` e os cabeçalhos de segurança/CSP no servidor do site.
8. Limite de corpo no proxy (ex.: Nginx `client_max_body_size 12m`) e timeout menor para uploads.
9. `UPLOADS_DIR` em volume persistente (com backup antes de ligar `UPLOADS_CLEANUP_INTERVAL_HOURS`), `APP_WEB_URL` e `PUBLIC_API_URL` reais e loja padrão definida (se o app mobile estiver publicado). `DB_POOL_SIZE` compatível com o `max_connections` do PostgreSQL.

10. E-mail por SMTP (`MAIL_DRIVER=smtp`, `SMTP_*`, `MAIL_FROM` com SPF/DKIM no domínio) conferido pelo e-mail de teste do superpainel; sem isso, convites e links de redefinição não chegam.
11. `HOST=127.0.0.1` quando o proxy reverso roda na mesma máquina (`0.0.0.0` só se a API precisa ser alcançada por outra máquina). Depois de atualizar, "Regravar credenciais" do Mercado Pago até não restar nenhuma no formato antigo.
12. Cobrança SaaS em `MANUAL` até validar a conta Mercado Pago da plataforma.
13. Log da subida sem avisos de configuração.

Checklist comentado: `docs/saas/operacao.md`, seção 9. Com Docker, os cabeçalhos do site, o rewrite da SPA e o `TRUST_PROXY=1` já vêm prontos: veja `docs/saas/docker.md` (produção e HTTPS).

---

22. Integrações externas

Integração	Situação	O que falta / observações
Mercado Pago (delivery, por loja)	Implementado	Credenciais de produção de cada loja; `MP_NOTIFICATION_URL`; nonce de CSP no servidor do site para o antifraude.
Mercado Pago Assinaturas (cobrança SaaS)	Implementado, não validado contra o MP real	Conta da Comestra, `MP_SAAS_ACCESS_TOKEN`, webhook `POST /billing/webhook/mercadopago` cadastrado com `MP_SAAS_WEBHOOK_SECRET`, `SAAS_BILLING_BACK_URL` e testes em sandbox.
E-mail transacional	SMTP implementado (nodemailer); padrão `log`	Servidor SMTP de produção (`MAIL_DRIVER=smtp`, `SMTP_*`, `MAIL_FROM`) testado pelo superpainel e `PUBLIC_API_URL` para o logo das lojas. Os e-mails já saem em texto e HTML com a marca. Evolução: fila de reenvio, provedores por API e bounces. Com SMTP, o superpainel deixa de mostrar os links de redefinição e de convite do detalhe da loja.
Domínio próprio por loja	Campo armazenado, sem roteamento	DNS curinga, certificados e resolução por host.
Rate limit distribuído	Em memória, por instância	Redis (ou similar) com várias instâncias.
ViaCEP / BrasilAPI	Opcional	Busca de CEP com reserva e conferência do bairro para o frete (cache por instância); sem os dois, o endereço é preenchido à mão e vale o bairro digitado.
football-data.org	Opcional	`FOOTBALL_DATA_TOKEN` para o banner de jogos.

---

23. Migração a partir do TorresBurgers

- **Banco:** o padrão agora é `comestra`. Para manter os dados, use `DB_DATABASE=torresburgers` (e `DB_PASSWORD=0000`, se a instalação dependia da senha padrão antiga) ou copie o banco com `createdb -T torresburgers comestra`.
- **Loja única → SaaS:** na primeira subida, as migrations movem cardápio, pedidos, pagamentos e equipe para a loja inicial (loja padrão), com um plano interno sem cobrança (seção 6). Faça o backup antes.
- **Sessões:** as chaves mudaram (web `torresburgers.*` → `comestra.*`; app `@TorresBurgers:*` → `@comestra:*`). Todos precisam entrar de novo.

- **Equipe:** `POST /users` sem token de ADMIN cria só clientes; a equipe entra por convite ou é criada pelo ADMIN da loja. Recuperação de acesso pelo superpainel (ou `npm run seed` em desenvolvimento).
- **.env antigo:** compare com o `backend/.env.example` (variáveis novas da plataforma: `CREDENTIALS_ENCRYPTION_KEY`, `DB_MIGRATIONS_RUN`, `TRUST_PROXY`, `SEED_DEMO_SUPERADMIN`, `JWT_LEGACY_ISSUED_BEFORE`, `PUBLIC_API_URL`, as da sessão em cookie e as do banco...).

24. Limitações conhecidas e pendências

Tema	Situação atual	Recomendação
Token de acesso	Site resolvido: token de 15 min em memória e refresh rotativo em cookie `HttpOnly`, com detecção de reuso e revogação da sessão substituída; tokens sem `jti` só na janela de `JWT_LEGACY_ISSUED_BEFORE`. O app mobile, a sessão de suporte e o site em outro domínio seguem com o token de 7 dias.	Sessão deslizante e limite de ociosidade; refresh no app mobile e logout ao sair; token compartilhado entre abas; CSRF double-submit; superpainel em origin próprio.
Sessões registradas	Sem "último acesso" por sessão; limpeza só quando uma sessão nova é emitida.	`last_seen_at` e limpeza global periódica pela manutenção.
Conta e credenciais da equipe	Confirmação e troca de e-mail em modo `compat` (o `strict` não existe ainda); ADMIN define senha/e-mail de membros; 409 em cadastro/convite com e-mail existente (o app mobile depende disso).	Fase 1b: respostas neutras, link de redefinição pelo ADMIN, equipe pendente até confirmar.
E-mail transacional	SMTP disponível (padrão `log`); texto e HTML com a marca; logo em WebP; uma falha não é reenviada; a criação do convite espera o envio.	Fila de reenvio, provedores por API, DKIM e bounces; logo em PNG; pré-visualização no superpainel; remetente por loja.
Webhook do delivery	Sem validação de `x-signature` (exige segredo por loja); contido por reconsulta, limite por IP e fila.	Segredo de webhook cifrado por loja e validação antes de consultar o MP.
Loja bloqueada	PIX pendentes cancelados pela conciliação em até 60 s por rodada; nenhum pedido nasce em loja bloqueada.	Cancelamento imediato nas ações de bloqueio.
Estorno pendente	Sem nova tentativa automática (alerta `REFUND_PENDING`).	Rotina com recuo exponencial e auditoria.
Preapproval SaaS	Assinatura suspensa pela plataforma continua sendo cobrada pelo MP (o pagamento não reativa). O checkout da assinatura chama o MP com uma conexão do banco aberta (no máximo uma por loja).	Pausar/retomar a preapproval nas ações de suspender/reativar; checkout com reserva por coluna e chamada fora da transação.
PIX e troco	PIX aceita até 30 dias; troco de até R$ 200 acima do total para todas as lojas.	Teto operacional para o PIX; teto de troco configurável por loja.
Limites	Em memória, por instância (CGNAT pode bloquear vários clientes atrás do mesmo IPv4); limites de produtos e equipe rígidos; o de pedidos do mês continua suave.	Redis com CAPTCHA ou credencial de dispositivo antes de escalar.

Tema	Situação atual	Recomendação
Arquivos	Limpeza manual ou agendada e cota por loja entregues; a cota usa cache por instância (até 5 min); a tela mostra só quantidade e tamanho; foto do cadastro sem prefixo de dono; avatares externos antigos no banco.	Contabilidade compartilhada do armazenamento, alerta no dashboard e script de saneamento dos avatares.
Frete por CEP	Zona pelo bairro do CEP e por faixas; o pedido mostra só o bairro digitado; sem reconferência periódica nem frete por distância.	Bairro do CEP no pedido, reconferência, frete por raio e lista pública dos bairros atendidos.
Desempenho	Índices e caches entregues; relatórios de um ano de lojas muito grandes ainda pesam (~0,5 s); histórico por `OFFSET`; auditoria sem retenção.	Agregar no SQL, paginação por cursor, retenção da auditoria e caches compartilhados.
Docker	O banco recebe o `.env.docker` inteiro e a API conecta como superusuário.	Segredo próprio do banco e papel sem superusuário.
Exposição em desenvolvimento	`HOST` disponível, mas o padrão continua `0.0.0.0` (app mobile); `/health` só com `status`; logins demo aceitos de qualquer origem; a API não se recusa a subir sem `CORS_ORIGINS`.	Loopback por padrão em development quando o app não depender da rede local; recusar logins demo fora da rede local.
Exibição	Quadro com os 500 pedidos mais recentes; Meus Pedidos com os 100 mais recentes.	"Carregar mais" nas telas.
Clientes abusivos	Bloqueio por loja disponível, mas as cobranças pendentes do cliente na loja não são canceladas ao bloquear (PIX gerado antes e pago depois vira pedido); sem cancelamento em lote.	Cancelar as cobranças pendentes ao bloquear e cancelamento em lote auditado.
Número do pedido	Sem busca por número no painel; respostas de pagamento sem o número; app mobile mostra o id.	`GET /orders?number=`, número nas respostas de pagamento e no app.

25. Troubleshooting

A API não conecta no banco

- Confirme que o PostgreSQL está rodando (Windows: serviço `postgresql-x64-<versão>`; Linux: `systemctl status postgresql`) e que o banco existe (`psql -U postgres -l`).
- Revise `DB_HOST`, `DB_PORT`, `DB_USERNAME`, `DB_PASSWORD` e `DB_DATABASE`. No Linux a conexão é TCP: o usuário precisa de senha (ou de `trust` no `pg_hba.conf`).

A API não sobe

- "JWT_SECRET não definido...": defina um `JWT_SECRET` de 32+ caracteres (obrigatório em produção).
- "Migration recusada: ... com papel global (platform_role) também pertence(m) à equipe...": remova o papel global (`--revoke`) ou tire a conta da equipe e suba de novo.

- Migration de loja única recusada com a lista de colunas faltando: banco de uma versão muito antiga; suba uma vez a versão anterior com `DB_SYNCHRONIZE=true` (com backup) e rode as migrations de novo.
- "A API não conseguiu escutar em <host>:<porta> (EADDRINUSE / EADDRNOTAVAIL...)": porta ocupada ou `HOST` que não existe nesta máquina. Confira `HOST` e `PORT` (um shell pode exportar `HOST` com o nome da máquina).

Contas e acesso

- Contas demo não entram: `cd backend && npm run seed` (só dev/test) e confira se `SEED_DEMO_USERS` não está `false`.
- Superadmin sem acesso: `npm run platform:superadmin -- --email=<e-mail> --reset-password`.
- "Esta loja foi desativada na plataforma.": a equipe de loja desativada não entra; restaure a loja no superpainel.
- "Sessão de suporte é somente leitura...": o token de suporte não altera nada; encerre o suporte e use uma conta da loja.
- "A sessão mudou em outra aba...": outra aba trocou de conta ou saiu; a aba atual acompanha a sessão do navegador.
- Todos precisaram entrar de novo depois de atualizar: os tokens antigos (sem `jti`) são recusados em produção sem `JWT_LEGACY_ISSUED_BEFORE` (defina-o com o instante do deploy na próxima vez). Em development, sem `JWT_SECRET`, o segredo passou a ser aleatório por máquina: entre de novo uma vez.
- Ao recarregar a página do site, a sessão some: confira se o login trouxe o cookie (`comestra_rt` ou `__Host-comestra_rt`) e se `POST /auth/web/refresh` responde 200; em produção o site precisa de HTTPS e estar no mesmo site da API (`CORS_ORIGINS` / `APP_WEB_URL` exatos). Fora disso, o site usa o modo bearer, que também funciona.
- "Sessão inválida ou expirada. Faça login novamente.": a sessão foi encerrada (sair, "Sessões ativas", "Sair de todos os aparelhos", superpainel, teto de 100 sessões, reuso de refresh) ou venceu. "A sessão de suporte terminou ou expirou...": inicie outra pelo superpainel.
- Login responde 500 de vez em quando: a sessão não pôde ser gravada (banco indisponível); nenhum token foi entregue, tente de novo.
- Convite, link de redefinição ou confirmação de e-mail não chegou: confira o driver em Superpainel → Configurações → E-mail transacional e envie um e-mail de teste; com o driver de log nada é enviado (copie o link exibido no painel). Confira também os limites: 3 links de "esqueci a senha" por conta em 24 h, 3 e-mails de conta por hora por destinatário e, nos convites, 10 por destinatário em 24 h somando as lojas (acima disso o convite sai sem e-mail).
- Endereço recusado com "Este endereço está fora da área de entrega da loja.": a loja recusa endereços fora das zonas; o bairro usado é o do CEP. Revise as zonas (nomes dos Correios ou faixas de CEP) ou escolha a retirada.

A web não acessa a API

- Abra `http://localhost:3000/health`, confira `VITE_API_URL` e reinicie o `npm run dev` depois de mudá-la.
- Com `CORS_ORIGINS` preenchido, inclua a origem do site.
- No build de produção, chamadas bloqueadas pela CSP indicam um build gerado com outra `VITE_API_URL`: gere o build de novo com a URL real.

Pedido recusado

- "A loja está fechada no momento.": abra a loja no painel (Dashboard ou Pedidos).

- "Esta loja está temporariamente indisponível para novos pedidos.": a loja está suspensa ou a assinatura não permite vender; veja o motivo no painel da loja ou no superpainel.
- "O produto "X" não aceita adicionais.": marque os adicionais permitidos no cadastro do produto.
- "Valor para troco acima do permitido...": informe até R$ 200 acima do total.
- 429 em pedidos, pagamentos ou endereços: limite de abuso; atrás de proxy, configure `TRUST_PROXY` para os limites por IP não juntarem todos os clientes.
- "Contas da administração da plataforma não fazem pedidos..." ou "Somente clientes fazem pedidos...": entre com uma conta de cliente.
- "Não é possível fazer pedidos nesta loja. Fale com a loja.": o cliente foi bloqueado por essa loja (Painel → Clientes → filtro Bloqueados); só o administrador da loja desbloqueia.
- Pedido recusado por chave duplicada em `UQ_orders_store_number` depois de restaurar ou importar pedidos: o contador da loja ficou para trás. Corrija com o SQL de `docs/saas/operacao.md` (seção 8.3), que leva `store_order_counters` ao maior número de cada loja.

Pagamento online

- 503 "Esta loja ainda não aceita pagamento online...": a loja não tem credenciais Mercado Pago utilizáveis (Configurações → Pagamentos) ou, em produção, `CREDENTIALS_ENCRYPTION_KEY` não está definida ou é inválida. O pagamento na entrega continua funcionando.
- "Você já tem pagamentos pendentes nesta loja...": pague ou cancele um PIX pendente.
- "Pagamento não encontrado ou expirado...": o `paymentId` não existe, venceu ou é de outro cliente ou loja.
- Cartão não carrega na web: o SDK do Mercado Pago precisa de internet e não é carregado para contas da equipe ou da plataforma.

Upload falha

- Um único arquivo no campo `file`, JPG, PNG ou WebP de até 5 MB e até 40 megapixels.
- 429: limite de uploads por conta (multipart malformado também conta). Verifique também a permissão de escrita em `UPLOADS_DIR`.
- 403 "Limite de armazenamento do plano ... atingido": a loja chegou ao `max_storage_mb` do plano. Aumente o limite ou troque o plano no superpainel, ou rode a limpeza dos arquivos sem uso.
- 400 "Formulário inválido: nome de campo não aceito.": nome de campo multipart aninhado demais ou com índice acima de 300.

Testes E2E não sobem

- Informe `DB_PASSWORD` no ambiente (o reset do banco não lê o `backend/.env`) e confira se o Google Chrome está instalado.
- Portas 3001/5174 ocupadas: libere-as ou use `E2E_API_PORT` / `E2E_WEB_PORT`. Mudou o backend? Pare a API de teste para o `npm test` recompilar. Compilar de novo numa pasta `dist-e2e-*` própria falha com TS5055: apague a pasta antes.

Docker

- `dependency failed to start: container comestra-api-1 is unhealthy`: veja `docker compose logs api` (`JWT_SECRET` vazio ou curto em produção, senha do banco diferente da gravada no volume ou migration com erro).
- Porta 8080 ocupada: `COMESTRA_HTTP_PORT=8081 docker compose up -d` e ajuste `APP_WEB_URL`, `CORS_ORIGINS` e `PUBLIC_API_URL`. Outros casos em `docs/saas/docker.md`, seção 11.